

Section 6: Evaluation and Monitoring (12%) – MCQ Practice Set

60 Questions | Difficulty: Beginner → Proficiency | Exam: Databricks Certified Generative AI Engineer Associate

Question 1

Difficulty: Beginner

A developer wants to evaluate whether a deployed RAG chatbot's answers are supported by the retrieved documents, without having any pre-written reference answers. Which MLflow built-in scorer should they use?

- **A) AnswerCorrectness** — this scorer compares the model's answer against a known correct reference answer stored in the golden dataset to determine factual accuracy.
- **B) RetrievalGroundedness** — this scorer evaluates whether the model's answer is supported by the retrieved context documents, comparing the output to the retrieved chunks, NOT to a reference answer.
- **C) AnswerSimilarity** — this scorer measures semantic similarity between the model's output and a reference answer, quantifying how close the generated text is to the expected response.
- **D) ExactMatch** — this scorer checks whether the model's output exactly matches the expected string in the evaluation dataset, character by character.

Correct Answer: B **Explanation:** B is correct. `RetrievalGroundedness` is specifically designed for scenarios where you want to verify that the model's response contains only claims supported by the retrieved documents — without needing a human-written reference answer. It compares the MODEL'S OUTPUT to the RETRIEVED CONTEXT CHUNKS (not to a ground truth). A grounded response does not introduce facts beyond what the retrieved documents contain. This is the key metric for detecting RAG hallucinations. A is wrong because `AnswerCorrectness` REQUIRES ground truth (a known correct reference answer) — you cannot run it without a pre-written expected output in the evaluation dataset. C is wrong because `AnswerSimilarity` also REQUIRES ground truth — it measures semantic closeness between the model output and a reference text. D is wrong because `ExactMatch` is a code-based scorer that REQUIRES an expected output string to compare against; it also requires ground truth. **Source:** Section 6: Evaluation and Monitoring – Objective 6: Identify evaluation judges that require ground truth — docs.databricks.com (search: "MLflow built-in scorers genai")

Question 2

Difficulty: Beginner

What is the primary difference between offline evaluation and online monitoring of an LLM application?

- **A)** Offline evaluation uses GPU-based compute while online monitoring uses CPU-only compute — the hardware difference determines which type of analysis is possible in each context.
- **B)** Offline evaluation assesses application quality BEFORE deployment using curated test datasets ("Is this ready to ship?"). Online monitoring tracks a deployed application's behavior in production using live traffic data ("Is this still performing well?"). They answer different questions at different stages of the lifecycle.
- **C)** Offline evaluation is always automated (using MLflow scorers), while online monitoring always requires human reviewers. The staffing model is the distinguishing factor between the two approaches.
- **D)** Offline evaluation is performed by the development team, while online monitoring is performed by the operations team — the organizational responsibility determines which category an evaluation activity falls into.

Correct Answer: B **Explanation:** B is correct. The offline vs. online distinction maps to two different lifecycle phases: **Offline evaluation** happens PRE-DEPLOYMENT. It uses a curated golden dataset with known inputs/outputs and automated scorers (MLflow `evaluate()`). The goal is to answer "Is this agent good enough to deploy?" — a quality gate decision. Tools: `mlflow.genai.evaluate()`, built-in scorers (RelevanceToQuery, RetrievalGroundedness), custom scorers, SME review. **Online monitoring** happens POST-DEPLOYMENT. It uses live production traffic — real user queries with real agent responses. The goal is to answer "Is the deployed agent still performing well? Has quality degraded?" — a continuous quality assurance function. Tools: Inference Tables, Agent Monitoring, Usage Tables, rate limiting. A is wrong because the hardware used (GPU vs. CPU) has nothing to do with offline vs. online classification — both can use GPUs (e.g., LLM judges run on GPU for both). C is wrong because online monitoring can also be automated (Agent Monitoring automatically applies LLM judges to production traffic); and offline evaluation can include human review (MLflow Review App). D is wrong because organizational responsibility (dev team vs. ops team) is not the defining characteristic — many teams do both.

Source: Section 6: Evaluation and Monitoring – Objectives 1 & 5: LLM selection and inference table tracking — docs.databricks.com (search: "Mosaic AI Agent Monitoring")

Question 3

Difficulty: Beginner

A developer is comparing two LLMs for a coding assistant application. They want to know which model is better at generating Python functions. Which academic benchmark is most relevant for this initial shortlisting?

- **A)** MMLU (Massive Multitask Language Understanding) — this benchmark tests general knowledge across 57 subjects including computer science, making it the best predictor of Python code generation quality.
- **B)** MT-Bench — this benchmark specifically measures multi-turn conversational coding ability, including Python function generation across 10 difficulty levels.
- **C)** HumanEval — this benchmark was specifically designed to measure code generation ability (Python function completion), making it the most directly relevant benchmark for selecting a coding assistant model.
- **D)** GSM8K — this benchmark tests multi-step mathematical reasoning, and since programming often involves algorithmic thinking, GSM8K scores correlate strongly with Python code generation quality.

Correct Answer: C **Explanation:** C is correct. HumanEval is the OpenAI-designed benchmark specifically built to measure code generation quality. It presents models with Python function signatures and docstrings, requiring the model to generate a working function implementation. The score (pass@k) measures the percentage of problems where the model generates a correct solution. For selecting a coding assistant LLM, HumanEval directly measures the relevant capability. A is wrong because MMLU tests broad general knowledge (math, law, ethics, science, history) — while it includes computer science questions, these are theoretical knowledge questions, not practical code generation tasks. High MMLU scores don't necessarily translate to better Python function generation. B is wrong because MT-Bench evaluates multi-turn conversation quality and instruction following for general chat — it is not specifically a coding benchmark and does not include Python function generation tasks. D is wrong because GSM8K tests grade-school level mathematical word problems (arithmetic reasoning) — while mathematical reasoning and programming are related skills, GSM8K scores do not reliably predict code generation quality for Python functions. **Source:** Section 6: Evaluation and Monitoring – Objective 1: Select an LLM based on quantitative metrics — [docs.databricks.com \(search: "MLflow evaluate generative AI scorers"\)](https://docs.databricks.com/search/MLflow+evaluate+generative+AI+scorers)

Question 4

Difficulty: Beginner

A company wants to prevent any single team from consuming more than 50,000 tokens per hour on their shared LLM endpoint. Which Databricks feature implements this control?

- **A)** Provisioned Throughput autoscaling — configure the endpoint's max tokens/second to $50,000/3600 \approx 14$ tokens/second, which limits each team to 50,000 tokens per hour through capacity restriction.
- **B)** Unity AI Gateway Rate Limiting — configure per-user or per-Service Principal token quotas in the Gateway settings. When a user or team's Service Principal hits the 50,000 token/hour limit, the Gateway returns HTTP 429 (Too Many Requests).
- **C)** MLflow experiment budget tracking — set a token budget parameter in `mlflow.start_run()` that automatically stops the run when the team's hourly token consumption exceeds 50,000.
- **D)** Delta table row-count triggers — configure a Databricks workflow that monitors the Inference Table row count and stops the serving endpoint when the count reaches the hourly limit.

Correct Answer: B **Explanation:** B is correct. Unity AI Gateway Rate Limiting is the correct tool for enforcing per-user, per-Service Principal, or per-team token consumption quotas. Configuration is done in the Gateway UI for the serving endpoint: set token-based quotas (e.g., 50,000 tokens/hour per Service Principal). When a team's Service Principal hits the limit, the Gateway returns HTTP 429 without forwarding the request to the LLM — preventing cost overruns and ensuring fair resource allocation across teams. This requires no application code changes — the enforcement is at the infrastructure level. A is wrong because Provisioned Throughput max tokens/second is a capacity configuration for how many tokens the endpoint can serve simultaneously — it is not a per-user quota mechanism. Reducing max throughput affects ALL users equally and doesn't track per-team consumption. C is wrong because `mlflow.start_run()` is for tracking ML experiments (training, evaluation), not for enforcing production inference token quotas — MLflow does not have a "token budget" parameter that stops inference operations. D is wrong because monitoring row counts in Inference Tables is a monitoring approach (retroactive), not a real-time enforcement mechanism; stopping the serving endpoint entirely would affect all users, not just the team that exceeded their quota. **Source:** Section 6: Evaluation and Monitoring – Objective 4: Use Databricks features to control LLM costs — [docs.databricks.com](https://docs.databricks.com/search?q=Unity+AI+Gateway+rate+limiting) (search: "Unity AI Gateway rate limiting")

Question 5

Difficulty: Beginner

Which MLflow function enables automatic tracing of a LangChain-based agent, capturing every LLM call, tool invocation, and retrieved document in a structured span tree?

- **A)** `mlflow.langchain.log_model()` — this function logs the LangChain model artifact to the MLflow registry and automatically enables production tracing for all requests handled by the registered model.
- **B)** `mlflow.langchain.autolog()` — this function enables automatic tracing for LangChain agents, capturing every LLM call, chain step, tool invocation, and retrieved document as a structured trace with timing data.
- **C)** `mlflow.set_tracking_uri("databricks")` — this configuration function routes all MLflow tracking data (including traces) to the Databricks workspace backend, enabling structured span capture for any Python application.
- **D)** `mlflow.start_run(tags={"trace": "full"})` — this context manager starts an MLflow run with full tracing enabled, recording all Python function calls made within the `with` block as trace spans.

Correct Answer: B **Explanation:** B is correct. `mlflow.langchain.autolog()` is the MLflow integration function that automatically instruments LangChain agents for tracing. When called at the start of a script or notebook, it patches LangChain's internal chain execution to capture every step as a span: the top-level chain invocation, individual LLM calls (with prompt/response), tool invocations (with inputs/outputs), retriever calls (with query and returned documents), and their durations. The resulting trace is viewable in the MLflow UI under Experiments → Traces, providing full visibility into agent execution without requiring manual `@mlflow.trace` decorators. A is wrong because `mlflow.langchain.log_model()` logs the model artifact to MLflow registry (for deployment) — it does not enable request-level tracing of individual inference calls. C is wrong because `mlflow.set_tracking_uri()` specifies WHERE MLflow metadata is stored — it does not enable automatic tracing of LangChain operations. D is wrong because `mlflow.start_run()` creates an experiment run for logging metrics/parameters — the `tags` parameter is for metadata labels, not for enabling execution tracing.

Source: Section 6: Evaluation and Monitoring – Objective 2: Use MLflow and Agent Framework for scoring and tracing — docs.databricks.com (search: "MLflow Tracing agent Databricks")

Question 6

Difficulty: Beginner

A production RAG endpoint has Inference Tables enabled. A support engineer wants to find all requests in the last 7 days that took longer than 5,000 milliseconds. Which SQL query correctly retrieves this information?

- **A)** `SELECT * FROM system.serving.endpoint_usage WHERE latency_ms > 5000 AND date >= CURRENT_DATE - 7;`
- **B)** `SELECT request_id, latency_ms, status_code FROM main.monitoring.my_rag_endpoint_payload_logs WHERE date >= CURRENT_DATE - 7 ORDER BY latency_ms DESC LIMIT 100;`
- **C)** `SELECT request_id, latency_ms FROM mlflow.traces WHERE execution_time_ms > 5000 AND created_at >= CURRENT_DATE - 7;`
- **D)** `SELECT * FROM main.monitoring.my_rag_endpoint_payload_logs WHERE latency_ms > 5000 AND date >= CURRENT_DATE - 7;`

Correct Answer: D **Explanation:** D is correct. Inference Tables log request payload data (inputs, outputs, latency, status codes) to a Unity Catalog Delta table in the schema chosen at setup. The correct query pattern: (1) Query the `payload_logs` table in the configured Unity Catalog location (not `system.serving.endpoint_usage` which is for token cost tracking). (2) Filter on `date >= CURRENT_DATE - 7` for the time window. (3) Filter on `latency_ms > 5000` for the latency threshold. Query D correctly targets the inference payload logs table and applies both filters. A is wrong because `system.serving.endpoint_usage` is the USAGE TABLE for token cost tracking (total tokens, request counts per endpoint/user) — it does not contain individual request latency data. C is wrong because there is no `mlflow.traces` SQL table — MLflow traces are viewable in the MLflow UI and accessible via the MLflow Python API (`mlflow.search_traces()`), not via a standard SQL table with `execution_time_ms`. B is only partially correct — it queries the right table but does NOT filter on `latency_ms > 5000`, so it would return ALL requests sorted by latency, not just the slow ones. **Source:** Section 6: Evaluation and Monitoring – Objective 3: Use inference logging to assess deployed RAG application performance — docs.databricks.com (search: "Inference tables Databricks Model Serving")

Question 7

Difficulty: Beginner

What is the MLflow Review App, and who is it designed for?

- **A)** The MLflow Review App is a developer tool that provides a code diff viewer for comparing MLflow model versions — it shows which parameters, code, and dependencies changed between two registered model versions, enabling technical review before promotion.
- **B)** The MLflow Review App is a Databricks-hosted web UI designed for Subject Matter Experts (SMEs) and non-technical human evaluators. It allows them to interact with agents, rate responses (thumbs up/down), and annotate production traces with expected outputs — without requiring any Python coding skills.
- **C)** The MLflow Review App is an automated quality review pipeline that runs MLflow `evaluate()` against the production inference table every 24 hours and emails a quality report to the development team — the "app" refers to the automated review schedule.
- **D)** The MLflow Review App is a mobile application (iOS/Android) that allows business stakeholders to approve or reject model deployment decisions from their phones, integrating with the Databricks MLflow Model Registry approval workflow.

Correct Answer: B **Explanation:** B is correct. The MLflow Review App is specifically designed for SMEs (Subject Matter Experts) and human evaluators who need to assess agent quality but are not ML engineers or Python developers. It is a web-based interface hosted by Databricks that provides two modes: (1) **Interactive Chat Mode** — the SME directly converses with the agent, testing edge cases and domain-specific scenarios, then rates each response with thumbs up/down and written feedback. (2) **Trace Labeling Mode** — the SME reviews historical production traces, annotating them with quality ratings and "expected output" corrections. The SME's feedback is automatically stored as MLflow Assessments, attached to the specific traces they evaluated, without the SME needing to write any code. A is wrong because the MLflow Review App is not a code diff viewer — that functionality is in version control tools; MLflow does track model parameters but doesn't provide UI-based code review. C is wrong because the Review App is interactive (human-driven), not an automated 24-hour pipeline — automated quality scoring uses Agent Monitoring and `mlflow.genai.evaluate()`. D is wrong because there is no MLflow Review App mobile application — it is a web browser-based tool. **Source:** Section 6: Evaluation and Monitoring – Objective 9: Incorporate SME feedback to improve agent performance — docs.databricks.com (search: "MLflow Review App human feedback")

Question 8

Difficulty: Beginner

A developer builds a financial chatbot and wants to verify that every response includes the phrase "not financial advice" OR "consult a professional." Which type of custom scorer is most appropriate?

- **A)** A Custom LLM Judge using `make_genai_metric()` — configure the LLM to evaluate whether the response has "the right regulatory tone and compliance disclosures" using a qualitative prompt, scoring 1–5 based on how thoroughly it addresses legal disclaimers.
- **B)** A Code-Based Custom Scorer using the `@scorer` decorator — implement a Python function that checks for the presence of the required disclaimer phrases using string matching, returning 1.0 if found and 0.0 if missing.
- **C)** The built-in `Safety` scorer — the Safety scorer automatically detects missing legal disclaimers in financial contexts and flags non-compliant responses as safety violations.
- **D)** The built-in `Guidelines` scorer — the Guidelines scorer performs exact string matching for specified required phrases in any response, returning a binary pass/fail result for compliance requirements.

Correct Answer: B **Explanation:** B is correct. Checking for the presence of specific required phrases ("not financial advice," "consult a professional") is a DETERMINISTIC task — either the phrase is present or it is not. This is precisely the use case for a Code-Based Custom Scorer. The `@scorer` decorator wraps a Python function that receives the inputs and outputs and returns a score. Implementation: `text = outputs["response"].lower(); score = 1.0 if any(kw in text for kw in ["not financial advice", "consult a professional"]) else 0.0`. This approach is: fast (sub-millisecond, no LLM inference needed), deterministic (same input always produces same output), cost-free (no LLM API calls), and perfectly suited for exact/near-exact rule-based checks. A is wrong because using a Custom LLM Judge for this task is expensive (requires LLM inference), slow, and non-deterministic — LLM judges introduce variance for what is essentially a simple string search problem. C is wrong because the built-in `Safety` scorer evaluates content for harmful material (violence, hate speech, etc.) — it does not check for the presence of financial disclaimer phrases. D is wrong because the built-in `Guidelines` scorer evaluates whether responses follow qualitative style/tone guidelines (e.g., "be professional and concise") — it does not perform exact phrase presence checking and is not designed for compliance keyword detection.

Source: Section 6: Evaluation and Monitoring – Objective 8: Use Databricks custom scorers for evaluating agents and LLMs — docs.databricks.com (search: "MLflow custom scorers genai evaluate")

Question 9

Difficulty: Beginner

What does TTFT (Time to First Token) measure, and why is it critical for user-facing applications?

- **A)** TTFT measures the total time for the LLM to generate a complete response from start to finish — it represents the total latency a user experiences from submitting a question to reading the complete answer.
- **B)** TTFT (Time to First Token) measures the time from when a request is sent until the model begins streaming its FIRST output token. It is critical for user-facing applications because users perceive the chatbot as "fast" if text starts appearing quickly, even if the total response time is longer.
- **C)** TTFT measures the time for the first token to be retrieved from the Vector Search index during RAG document retrieval — it represents the retrieval latency component of the total RAG pipeline latency.
- **D)** TTFT measures the number of tokens in the first sentence of the model's response — it is used to evaluate whether the model produces concise, direct responses that get to the point quickly rather than using lengthy preambles.

Correct Answer: B **Explanation:** B is correct. TTFT (Time to First Token) is specifically the latency from when the API request is sent until the model starts streaming the first output token. For streaming-enabled model serving endpoints (which send tokens as they are generated, not all at once), TTFT determines how quickly the user sees ANY text response begin to appear. In human perception, a chatbot that starts responding in under 1 second feels "instant," while one that takes 3+ seconds before any text appears feels "slow" — even if both complete at the same total time. This is why TTFT is a critical user experience metric distinct from total response latency. A is wrong because TTFT is NOT the total generation time — total latency measures the end-to-end time for the complete response; TTFT is only the time to the START of streaming, which is typically much shorter. C is wrong because TTFT is an LLM streaming metric, not a Vector Search retrieval metric — retrieval latency is typically measured separately as part of the RAG pipeline trace. D is wrong because TTFT stands for Time to First TOKEN (a timing metric in milliseconds), not the number of tokens in the first output sentence. **Source:** Section 6: Evaluation and Monitoring – Objective 1: Select an LLM based on quantitative metrics — docs.databricks.com (search: "Foundation Model APIs model comparison")

Question 10

Difficulty: Beginner

What is an "Assessment" in the context of MLflow's human feedback system?

- **A)** An Assessment is an automated MLflow scorer output — it represents the numerical score that a built-in judge like `RelevanceToQuery` assigns to a single model response during `mlflow.evaluate()` .
- **B)** An Assessment is the stored record of a human evaluator's feedback on a specific trace. It contains: a rating (thumbs up/down or 1–5 score), an optional explanation comment, and optionally the "expected output" the evaluator thinks the correct answer should have been.
- **C)** An Assessment is a Unity Catalog access control audit entry — it records when a user accessed a specific table or model, used as the primary mechanism for GDPR compliance documentation.
- **D)** An Assessment is a Databricks MLflow Model Registry stage assignment — when a reviewer "assesses" a model version as ready for production, the model's stage transitions from "Staging" to "Production" in the Registry.

Correct Answer: B **Explanation:** B is correct. In the MLflow human feedback framework, an Assessment is the structured data object that stores a human evaluator's (or SME's) review of a specific trace. Created through the MLflow Review App (or programmatically), an Assessment is attached to a specific trace by its trace ID and contains: (1) **Rating** — a quantitative judgment (thumbs up/down, 1–5 score) on the overall quality or a specific dimension. (2) **Comment** — an optional free-text explanation of the rating ("The model gave the wrong dosage calculation"). (3) **Expected output** — the evaluator's correction of what the right answer should have been ("The correct dosage is 25mg/kg"). Assessments accumulate over time as SMEs review production traces through the Review App, creating a dataset of human-labeled quality judgments that can be used to: (a) identify failure patterns, (b) build new evaluation datasets, (c) fine-tune custom LLM judges to match SME judgment. A is wrong because Assessment is specifically a HUMAN feedback construct — automated scorer outputs are logged as `metric` values in the evaluation results, not as Assessments. C is wrong because Unity Catalog audit entries are a governance feature, not MLflow Assessments. D is wrong because model stage assignments in MLflow Registry (Staging, Production) are not called Assessments — they are model version aliases or stage transitions. **Source:** Section 6: Evaluation and Monitoring – Objective 9: Incorporate SME feedback — docs.databricks.com (search: "MLflow Review App human feedback")

Question 11

Difficulty: Beginner

Which MLflow scorer **REQUIRES** a ground truth (expected reference answer) to calculate its metric?

- **A)** `RetrievalGroundedness` — requires the reference answer to compare whether the retrieved documents contain the same facts as the expected output.
- **B)** `RelevanceToQuery` — requires the reference query to compare whether the model's response addresses the same topic as the original question in the golden dataset.
- **C)** `AnswerCorrectness` — requires a known correct reference answer to compare against the model's output, determining whether the generated answer is factually accurate.
- **D)** `Safety` — requires a safety policy reference document that defines which content categories are harmful, enabling comparison of the model's output against the policy.

Correct Answer: C **Explanation:** C is correct. `AnswerCorrectness` requires ground truth because it asks: "Is the model's answer factually CORRECT?" — and the only way to determine correctness is to compare the model's output to a KNOWN CORRECT ANSWER. Without a reference answer, the scorer cannot determine whether the model's response is right or wrong. The evaluation dataset must include an `expected_output` column (the reference answer) for `AnswerCorrectness` to function. A is wrong because `RetrievalGroundedness` measures whether the answer is SUPPORTED BY THE RETRIEVED CONTEXT (the retrieved documents) — it compares the model's output to the retrieved chunks, NOT to a reference answer; no ground truth is needed. B is wrong because `RelevanceToQuery` measures whether the response addresses the USER'S QUESTION — it compares the model's output to the INPUT QUERY only; no reference answer is needed. D is wrong because `Safety` evaluates the content against harm categories (violence, hate speech) using a trained safety classifier — it does not compare against a reference answer; no ground truth is needed. **Source:** Section 6: Evaluation and Monitoring – Objective 6: Identify evaluation judges that require ground truth — docs.databricks.com (search: "MLflow built-in scorers genai")

Question 12

Difficulty: Beginner

What SQL table do you query to find out which users are consuming the most tokens on a Databricks Model Serving endpoint over the past 30 days?

- **A)** `main.monitoring.endpoint_payload_logs` — the Inference Table's payload logs include a `token_count` column for each request, which can be aggregated by user ID to find the highest consumers.
- **B)** `system.serving.endpoint_usage` — this Databricks system table tracks token consumption (input tokens, output tokens, total tokens) per endpoint per user/Service Principal, enabling cost attribution queries grouped by `principal_name`.
- **C)** `mlflow.experiment_metrics` — MLflow logs token consumption as a metric (`total_tokens`) for each model evaluation run, which can be queried by the user who started each run.
- **D)** `information_schema.endpoint_requests` — the Databricks workspace information schema includes token usage statistics that can be filtered by user and time period for cost reporting.

Correct Answer: B **Explanation:** B is correct. `system.serving.endpoint_usage` is the Databricks system table specifically designed for LLM cost attribution. It contains: `endpoint_name`, `principal_name` (the user or Service Principal making the calls), `input_tokens`, `output_tokens`, `total_tokens`, `request_count`, and `timestamp`. The correct query: `SELECT principal_name, SUM(total_tokens) FROM system.serving.endpoint_usage WHERE timestamp >= CURRENT_DATE - 30 GROUP BY principal_name ORDER BY SUM(total_tokens) DESC`. This enables per-user cost attribution, team budget reporting, and identification of runaway consumption patterns. A is wrong because Inference Tables (`payload_logs`) log the full request and response PAYLOAD (text content) — they may include token counts in some configurations, but `system.serving.endpoint_usage` is the authoritative, pre-aggregated source for token cost tracking. C is wrong because `mlflow.experiment_metrics` does not exist as a SQL table — MLflow logs training experiment metrics; it does not track production inference token consumption. D is wrong because `information_schema.endpoint_requests` is not a real Databricks SQL table — `information_schema` contains database/table metadata, not serving endpoint usage statistics. **Source:** Section 6: Evaluation and Monitoring – Objective 7: Use AI Gateway (Inference Tables, Usage Tables, and rate limiting) — [docs.databricks.com \(search: "system.serving endpoint usage tables"\)](https://docs.databricks.com/search:system.serving%20endpoint%20usage%20tables)

Question 13

Difficulty: Beginner

A developer deploys an agent using `agents.deploy()` from the Mosaic AI Agent Framework. What monitoring capabilities are automatically configured?

- **A)** Only error logging is configured automatically — `agents.deploy()` sets up error tracking (4xx/5xx responses) but requires manual configuration for quality metrics, token tracking, and inference tables.
- **B)** Inference tables (payload logging), MLflow tracing on the endpoint, and Agent Monitoring (quality score dashboard) are all automatically configured when deploying with `agents.deploy()` — the Agent Framework handles the full monitoring stack without additional setup steps.
- **C)** `agents.deploy()` creates the endpoint but does NOT configure any monitoring — monitoring requires separate setup: navigate to the endpoint UI, click "Inference Tables → Set Up," then manually enable Agent Monitoring from the Mosaic AI console.
- **D)** `agents.deploy()` automatically configures real-time alerting — when quality scores drop below a threshold, Databricks automatically sends email alerts to all workspace administrators without any additional configuration.

Correct Answer: B **Explanation:** B is correct. When you deploy an agent using `agents.deploy()` from the Mosaic AI Agent Framework, Databricks automatically configures the full monitoring stack as part of the deployment process: (1) **Inference Tables** — the endpoint is automatically connected to a Unity Catalog Delta table that logs all incoming requests and outgoing responses. (2) **MLflow Tracing** — the deployed agent's endpoint has tracing enabled, capturing the span tree for every inference (LLM calls, tool invocations, retriever calls with timing). (3) **Agent Monitoring** — the Mosaic AI Agent Monitoring dashboard is automatically set up to apply quality scorers (LLM judges) to production traffic and track quality metrics over time (groundedness, relevance, safety). This is a key benefit of using the Agent Framework — the monitoring infrastructure is "batteries included." A is wrong because `agents.deploy()` configures much more than error logging — it sets up the full three-layer monitoring stack. C is wrong because manual navigation to the endpoint UI for setup is required for CUSTOM deployments (not using Agent Framework) — Agent Framework deployments auto-configure. D is wrong because automated email alerting is not a built-in feature of `agents.deploy()` — alert configurations would require additional setup (e.g., Databricks Lakehouse Monitoring alerts). **Source:** Section 6: Evaluation and Monitoring – Objective 5: Use inference tables and Agent Monitoring — docs.databricks.com (search: "Mosaic AI Agent Monitoring")

Question 14

Difficulty: Beginner

What is the key advantage of the `@mlflow.trace` decorator over `mlflow.langchain.autolog()` for adding tracing to a RAG application?

- **A)** `@mlflow.trace` is faster than autolog — it uses a more efficient tracing protocol that reduces the overhead from 15ms per call with autolog to under 1ms with the decorator.
- **B)** `@mlflow.trace` provides more detailed traces for LangChain applications — it captures 3× more span attributes than autolog for the same LangChain chain execution.
- **C)** `@mlflow.trace` provides granular control over which specific functions are traced and how they are labeled (e.g., `span_type="RETRIEVER"` , `span_type="LLM"`) — it is used for custom code that doesn't use a supported framework (LangChain, LlamaIndex, etc.) or for adding traces to specific functions within a larger application.
- **D)** `@mlflow.trace` is the only tracing option that works in production Databricks Model Serving endpoints — `mlflow.langchain.autolog()` only works in development notebooks, not in deployed endpoints.

Correct Answer: C Explanation: C is correct. `@mlflow.trace` is a decorator that explicitly marks specific Python functions for tracing — giving the developer precise control: (1) Which functions are included in the trace (only decorated functions, not all code). (2) What `span_type` label each function gets (`"RETRIEVER"` , `"LLM"` , `"TOOL"` , `"CHAIN"`) — useful for the MLflow UI's trace visualization to correctly categorize spans. (3) Custom span attributes. This is the preferred approach for: custom retrieval functions not built with LangChain, custom Python code that calls LLMs directly via the `mlflow.deployments` client, or adding trace spans to specific utility functions within a larger pipeline. `mlflow.langchain.autolog()` is more convenient for LangChain-based agents (zero code changes) but it provides automatic (not custom) span labeling. A is wrong because both approaches have comparable tracing overhead — the performance difference is negligible; the choice is about control, not speed. B is wrong because autolog captures all LangChain steps automatically — the decorator doesn't capture more details for the same LangChain code. D is wrong because both autolog and `@mlflow.trace` work in both development notebooks and production serving endpoints. **Source:** Section 6: Evaluation and Monitoring – Objective 2: Use MLflow and Agent Framework for scoring and tracing — docs.databricks.com (search: "MLflow Tracing agent Databricks")

Question 15

Difficulty: Beginner

A developer compares two LLMs using `mlflow.evaluate()` with the following results:

Model	groundedness	relevance_to_query	latency_ms	cost_per_1k_tokens
Model A (70B)	0.91	0.88	2,100	\$0.80
Model B (7B)	0.87	0.85	480	\$0.06

For a customer-facing real-time chatbot with a response time SLA of under 1 second, which model should be selected and why?

- **A)** Model A — with groundedness 0.91 vs. 0.87 and relevance 0.88 vs. 0.85, the quality metrics are superior; for a customer-facing application, quality should always take precedence over latency and cost considerations.
- **B)** Model B — at 480ms latency it comfortably meets the 1-second SLA, while Model A at 2,100ms violates the SLA completely, making it non-deployable for this use case regardless of its quality advantage. The 4-point quality difference (0.87 vs. 0.91) may be acceptable given the constraint.
- **C)** Model A — the 2,100ms latency can be reduced by enabling Provisioned Throughput with a high token-per-second allocation, bringing Model A's latency within the 1-second SLA threshold while retaining its quality advantage.
- **D)** Neither model — both have quality scores below 1.0 (perfect), so neither meets the production quality threshold for a customer-facing application. The team should re-evaluate once a model achieves groundedness ≥ 0.95 .

Correct Answer: B **Explanation:** B is correct. This question tests the practical skill of multi-dimensional model selection. Model A at 2,100ms latency would fail the 1-second SLA on EVERY request — it simply cannot be deployed for this use case regardless of its quality advantage. Model B at 480ms satisfies the SLA with a 520ms safety margin. The quality trade-off (groundedness 0.87 vs. 0.91, relevance 0.85 vs. 0.88) is a 4-point difference — which may be perfectly acceptable for customer service. The MLflow experiment provides the data to make this decision objectively: Model B is the correct choice because it MEETS THE CONSTRAINT (SLA) with acceptable quality. A is wrong because "quality always takes precedence" is an invalid principle when a hard SLA constraint exists — a 2,100ms response makes the chatbot unusable regardless of quality. C is wrong because Provisioned Throughput affects throughput capacity, not per-request generation latency — Model A's 2,100ms latency is the model's inference time, which is determined by model size and compute; Provisioned Throughput doesn't reduce it to under 1 second for a 70B model. D is wrong because there is no "0.95 production threshold" — production deployment decisions are based on contextual requirements, and 0.87 groundedness may be perfectly acceptable for many applications. **Source:** Section 6: Evaluation and Monitoring – Objective 1: Select an LLM based on quantitative metrics — docs.databricks.com (search: "MLflow evaluate generative AI scorers" and "Foundation Model APIs model comparison")

Question 16

Difficulty: Beginner

A deployed RAG chatbot's Agent Monitoring dashboard shows that the average groundedness score has dropped from 0.89 to 0.71 over the past two weeks. What does this indicate, and what should the team investigate?

- **A)** A drop in groundedness score indicates that the model serving endpoint has reached its token-per-second capacity limit — the endpoint is throttling requests and returning truncated responses that appear less grounded because they are incomplete.
- **B)** A drop in groundedness score (from 0.89 to 0.71) indicates that the model is increasingly generating responses that contain claims NOT supported by the retrieved documents — the model is hallucinating more. Investigate: (1) whether the knowledge base content has drifted (new user queries not covered by existing documents), (2) whether the retriever is returning less relevant chunks (retrieval precision drop), (3) whether the model version changed.
- **C)** A drop in groundedness score indicates that the content safety filters are now blocking the most grounded responses — the safety classifier is over-triggering and rejecting well-sourced factual responses, causing lower average scores.
- **D)** A drop in groundedness score is expected behavior when user query volume increases — more queries naturally leads to lower average quality because the model cannot maintain quality at high throughput.

Correct Answer: B **Explanation:** B is correct. Groundedness measures whether the model's answers are supported by the retrieved context. A drop from 0.89 to 0.71 is a significant quality regression (–18 points) that indicates the model is increasingly making claims not found in the retrieved documents. Root cause investigation paths: (1) **Knowledge base drift** — users may be asking about topics not covered in the current knowledge base, causing the retriever to return loosely-related chunks, and the model to fill the gap with hallucinated content. (2) **Retrieval degradation** — the Vector Search index may be stale (not synced with new source Delta table content), causing retrieval of outdated or less relevant chunks. (3) **Model change** — a model version update may have changed its tendency to stay grounded to context. Agent Monitoring's time-series view helps identify WHEN the drop started, providing a temporal clue for root cause. A is wrong because rate limiting/throttling causes HTTP 429 errors and dropped requests — it does not cause responses to gradually become less grounded over time. C is wrong because content safety filters block or modify responses with harmful content — they don't lower the groundedness score of non-blocked responses. D is wrong because query volume does not inherently cause quality degradation — a well-scaled endpoint maintains consistent quality at high throughput. **Source:** Section 6: Evaluation and Monitoring – Objective 5: Use inference tables and Agent Monitoring — docs.databricks.com (search: "Mosaic AI Agent Monitoring")

Question 17

Difficulty: Beginner

What is the difference between **Provisioned Throughput** and **Pay-per-token** billing models for Databricks Foundation Model API endpoints, and when is each preferred?

- **A)** Provisioned Throughput is for development workloads (cheaper for low-volume testing), while Pay-per-token is for production workloads (scales automatically to any volume). The billing model determines maximum endpoint capacity.
- **B)** Provisioned Throughput reserves dedicated tokens-per-second capacity and charges hourly regardless of utilization — preferred for predictable, high-volume production loads where consistent low latency is required. Pay-per-token charges only for tokens consumed — preferred for variable or prototype workloads where traffic is unpredictable or low.
- **C)** Provisioned Throughput uses Databricks-managed LLMs only, while Pay-per-token supports both Databricks-managed and custom fine-tuned models. The billing model determines which model types are accessible.
- **D)** Provisioned Throughput uses dedicated single-tenant GPU hardware, eliminating noisy-neighbor latency variability, while Pay-per-token uses shared multi-tenant hardware. Both support the same models and traffic volumes.

Correct Answer: B **Explanation:** B is correct. The two billing models have distinct economic characteristics:

Provisioned Throughput: You reserve a specific token/second capacity (e.g., 5,000 tokens/sec). You pay an HOURLY rate based on the reserved capacity, whether you use it or not. Benefit: guaranteed capacity = consistent, predictable latency — no queuing delays during traffic spikes. Best for: steady-state production workloads with predictable traffic patterns where latency SLAs are critical. **Pay-per-token:** You pay only for the tokens you actually consume. No reserved capacity — requests are served from a shared pool. Benefit: zero idle cost — perfect for low-volume, variable, or prototype workloads. Risk: latency may vary during peak periods. Best for: development, testing, or production workloads with unpredictable or low traffic volumes. A is wrong because it reverses the recommendation — Provisioned Throughput is for production, not development; Pay-per-token is cost-effective for development. C is wrong because the billing model does not determine which model types are accessible — both billing modes support Databricks-managed and custom models. D is wrong because while dedicated hardware is a characteristic of Provisioned Throughput, the defining commercial difference is the billing model (hourly capacity vs. per-token consumption), not just the hardware. **Source:** Section 6: Evaluation and Monitoring – Objective 4: Use Databricks features to control LLM costs — docs.databricks.com (search: "Unity AI Gateway rate limiting")

Question 18

Difficulty: Beginner

An SME (Subject Matter Expert) doctor reviews 50 production traces of a medical RAG chatbot using the MLflow Review App. She rates 12 responses with thumbs down and provides corrections for each. Where is this feedback stored, and how does the development team access it?

- **A)** The feedback is stored as MLflow Experiment run parameters — each thumbs-down creates a new MLflow run tagged with `{"review_result": "negative"}`, accessible in the Experiments UI sorted by tag.
- **B)** The feedback is stored as MLflow Assessments — structured records attached to each reviewed trace by trace ID, containing the rating (thumbs down), optional comment, and optional expected output correction. Accessible via the MLflow API (`mlflow.get_assessments(trace_id=...)`) or the Review App's Assessments dashboard.
- **C)** The feedback is stored in a separate Databricks table specified by the SME at the start of the review session — the team must query this custom table by filtering for `rating = 'negative'` to retrieve the doctor's corrections.
- **D)** The feedback is temporarily stored in the MLflow Review App's session cache for 72 hours, then automatically deleted — the development team must export the data within 72 hours using the Review App's "Download Feedback" button before it is purged.

Correct Answer: B **Explanation:** B is correct. The MLflow Review App automatically stores each piece of human feedback as an **Assessment** object in MLflow's persistence layer. Each Assessment is: (1) Attached to the specific trace ID it evaluated (linking feedback to the exact agent execution being reviewed). (2) Contains: `rating` (thumbs up/down or 1–5), `comment` (the doctor's written explanation), and optionally `expected_output` (the doctor's correction of what the answer should have been). (3) Persisted indefinitely in the MLflow tracking server (not cached or auto-deleted). The development team accesses assessments through:

`mlflow.get_assessments(trace_id="...")` for individual traces, or by querying the assessments for all traces in an evaluation dataset to build a labeled improvement dataset. A is wrong because MLflow Experiment run parameters log model training configurations — they are not the storage mechanism for human feedback from the Review App. C is wrong because Assessments are stored in the MLflow backend automatically, not in a custom table specified by the SME — the SME simply reviews in the UI and the system handles storage. D is wrong because Assessments are permanently persisted in MLflow storage — there is no 72-hour cache expiration or forced export requirement. **Source:** Section 6: Evaluation and Monitoring – Objective 9: Incorporate SME feedback — docs.databricks.com (search: "MLflow Review App human feedback" and "Mosaic AI Agent Evaluation SME feedback")

Question 19

Difficulty: Beginner

A developer needs to evaluate an agent on a dataset where they do NOT have pre-written reference answers for any of the test inputs. They want to measure quality across three dimensions. Which combination of scorers can they use without any ground truth?

- **A)** `AnswerCorrectness` , `AnswerSimilarity` , `ExactMatch` — all three of these scorers can run without ground truth when the `expected_output` column is omitted from the evaluation dataset.
- **B)** `RelevanceToQuery` , `RetrievalGroundedness` , `Safety` — all three of these scorers evaluate quality without requiring a reference answer, comparing the model's output to either the input query, the retrieved context, or a safety policy.
- **C)** `AnswerCorrectness` , `RelevanceToQuery` , `Safety` — a mix of ground-truth-dependent and independent scorers; when ground truth is missing, `AnswerCorrectness` automatically falls back to an LLM-based heuristic evaluation.
- **D)** `AnswerSimilarity` , `RetrievalGroundedness` , `Guidelines` — answer similarity can operate without ground truth by comparing the model's output to a stylistically similar exemplar in the prompt.

Correct Answer: B **Explanation:** B is correct. The three scorers that do NOT require ground truth: (1)

`RelevanceToQuery` — evaluates whether the model's response addresses the user's question by comparing the OUTPUT to the INPUT QUERY. No reference answer needed. (2) `RetrievalGroundedness` — evaluates whether the model's response is supported by the RETRIEVED CONTEXT DOCUMENTS. No reference answer needed — it compares the output to the retrieved chunks. (3) `Safety` — evaluates the model's output for harmful content (violence, hate speech, adult content) against a safety classifier. No reference answer needed — it evaluates content, not accuracy. All three can run when the evaluation dataset only contains `inputs` and `outputs` (traces) without an `expected_output` column. A is wrong because `AnswerCorrectness` , `AnswerSimilarity` , and `ExactMatch` ALL require ground truth — there is no "automatic fallback to heuristic evaluation" for these scorers when ground truth is missing. C is wrong because `AnswerCorrectness` definitely REQUIRES ground truth — it does not fall back to heuristic evaluation when `expected_output` is absent. D is wrong because `AnswerSimilarity` requires ground truth (the reference answer to measure similarity against); there is no "stylistically similar exemplar" fallback mechanism. **Source:** Section 6: Evaluation and Monitoring – Objective 6: Identify evaluation judges that require ground truth — docs.databricks.com (search: "MLflow built-in scorers genai" and "MLflow evaluate ground truth required")

Question 20

Difficulty: Beginner

A company uses Databricks Model Serving for their LLM endpoint. They want to track which teams are spending the most money on LLM inference across the organization. What is the correct tool and query approach?

- **A)** Query `mlflow.experiments` to find the experiment with the highest token count in its logged metrics — each team creates their own MLflow experiment, so experiment-level token tracking provides team-level attribution.
- **B)** Query `system.serving.endpoint_usage` grouped by `principal_name` (the Service Principal or user associated with each team) and aggregate `total_tokens` — this system table provides token-level cost attribution at the team/user level.
- **C)** Navigate to the Databricks Account Console → Billing → Usage — the billing console provides per-team cost breakdown without requiring any SQL queries, directly showing LLM inference costs by organizational team.
- **D)** Enable Lakehouse Monitoring on each serving endpoint — Lakehouse Monitoring automatically tracks token consumption per team and generates daily cost reports broken down by department.

Correct Answer: B **Explanation:** B is correct. `system.serving.endpoint_usage` is the correct system table for token-level cost attribution. Each row represents token consumption for a specific request, with columns including `endpoint_name`, `principal_name` (the user or Service Principal making the call), `input_tokens`, `output_tokens`, and `total_tokens`. To track team-level spending: each team uses a dedicated Service Principal for their applications. Querying `GROUP BY principal_name` aggregates token consumption by team Service Principal, which can be correlated to a cost estimate using the per-token rate. This enables FinOps visibility — identifying which teams are the largest LLM consumers and should prioritize optimization. A is wrong because MLflow experiments track model training/evaluation runs, not production inference token consumption — there is no team-level token tracking in MLflow experiment metrics for production serving. C is wrong because while the Account Console provides billing information, it shows aggregate spending — it does not provide the per-principal attribution granularity available in `system.serving.endpoint_usage`. D is wrong because Lakehouse Monitoring is for data quality and model quality monitoring (detecting drift, data anomalies) — it does not automatically track token consumption per team or generate cost reports. **Source:** Section 6: Evaluation and Monitoring – Objectives 4 & 7: Control LLM costs and track via AI Gateway — docs.databricks.com (search: "system.serving endpoint usage tables" and "Unity AI Gateway rate limiting")

Question 21

Difficulty: Intermediate

A team runs `mlflow.genai.evaluate()` on their RAG pipeline with the following code:

```
results = mlflow.genai.evaluate(  
    data=eval_dataset,  
    predict_fn=my_rag_agent,  
    scorers=[  
        RelevanceToQuery(),  
        RetrievalGroundedness(),  
        AnswerCorrectness()  
    ]  
)
```

The `eval_dataset` only contains `inputs` (user queries) — it does NOT have an `expected_output` column. What happens when this code runs?

- **A)** All three scorers run successfully — when `expected_output` is absent, MLflow uses a self-consistency check (running the model twice and comparing outputs) as a substitute ground truth for `AnswerCorrectness`.
- **B)** `AnswerCorrectness` fails or produces null/invalid scores because it requires `expected_output` (ground truth) to compare against, while `RelevanceToQuery` and `RetrievalGroundedness` run successfully since they do not require ground truth.
- **C)** All three scorers fail — MLflow `evaluate()` requires the evaluation dataset to have both `inputs` and `expected_output` columns, otherwise the entire evaluation run raises a schema validation error before any scorers execute.
- **D)** The evaluation runs successfully but `AnswerCorrectness` uses the model's own output as its reference answer, resulting in a perfect score (1.0) for every row — this is a known bias in MLflow's evaluation when ground truth is missing.

Correct Answer: B Explanation: B is correct. MLflow scorers have explicit ground truth requirements:

`RelevanceToQuery` — requires only `inputs` and `outputs` (the model's response). Runs successfully.

`RetrievalGroundedness` — requires `inputs`, `outputs`, and the retrieved context (typically captured in the trace). Runs successfully. `AnswerCorrectness` — requires `inputs`, `outputs`, AND `expected_output` (the

reference correct answer). When `expected_output` is missing from the dataset, this scorer cannot compute a meaningful score — it will either raise an error or return null/NaN scores for all rows. The other two scorers are unaffected and produce valid results. The practical lesson: always verify which scorers you're using require ground truth before designing your evaluation dataset. If you only have traces (no expected outputs), use only ground-truth-free scorers. A is wrong because there is no "self-consistency check" fallback in MLflow —

`AnswerCorrectness` does not substitute a comparison mechanism when `expected_output` is absent. C is wrong because the entire evaluation does NOT fail — scorers that can run (`RelevanceToQuery`, `RetrievalGroundedness`) will still execute; the failure is isolated to `AnswerCorrectness`. D is wrong because using the model's own output as its reference would be circular — MLflow does not implement this behavior. **Source:** Section 6: Evaluation and Monitoring – Objective 6: Identify evaluation judges that require ground truth — docs.databricks.com (search: "MLflow built-in scorers genai" and "mlflow.genai.evaluate scorers")

Question 22

Difficulty: Intermediate

A developer builds a legal document RAG chatbot. They want to evaluate whether responses use formal legal language and avoid colloquial phrases. No ground truth answers exist. Which custom scorer type and implementation is most appropriate?

- **A)** A Code-Based Scorer (`@scorer`) that uses a regular expression to check for colloquial phrases like "gonna," "wanna," "kinda" — if any are found, it returns 0.0; otherwise 1.0.
- **B)** A Custom LLM Judge using `make_genai_metric()` — configure an LLM to evaluate whether the response uses formal legal language and is free of colloquialisms, with a grading prompt that defines formal vs. informal criteria, scoring 1–5.
- **C)** The built-in `Guidelines` scorer — configure it with the guideline text "Responses must use formal legal language without any colloquial phrases" — the Guidelines scorer automatically enforces text formality requirements.
- **D)** Both A and B are appropriate; use A for speed (high volume) and B for quality (sample of responses). The `@scorer` decorator efficiently detects obvious colloquialisms at scale, while the LLM judge catches subtle formality issues in a sample.

Correct Answer: D Explanation: D is correct, but it requires understanding the trade-offs: **Option A (Code-Based Scorer with regex)** — fast, deterministic, zero LLM cost, but LIMITED. A regex for "gonna, wanna, kinda" only catches the most obvious colloquialisms — it misses "it's kinda like...", "basically...", "super important," and other informal language. Good for a quick, high-volume pre-filter. **Option B (Custom LLM Judge)** — `make_genai_metric()` creates an LLM evaluator with a custom grading prompt that defines formality criteria. An LLM can evaluate nuanced formality ("somewhat informal due to use of contractions" vs. "appropriately formal legal register") — capturing what regex cannot. The trade-off is LLM inference cost (each scored response requires an LLM call). **Option C (built-in Guidelines scorer)** — the `Guidelines` scorer evaluates against natural-language guidelines using an LLM judge; this IS a valid approach similar to B. However, `make_genai_metric()` provides more control over the scoring rubric (1–5 scale with specific criteria). D is the most sophisticated and practical answer — use regex screening at scale for obvious violations and LLM judging for subtle quality assessment on a representative sample. However, if only ONE approach is allowed, B is more complete. The exam often asks about this layered approach for cost-effective evaluation. **Source:** Section 6: Evaluation and Monitoring – Objective 8: Use Databricks custom scorers — docs.databricks.com (search: "MLflow custom scorers genai evaluate" and "make_genai_metric Databricks")

Question 23

Difficulty: Intermediate

A data engineering team wants to run a nightly enrichment job that classifies 2 million customer support tickets using an LLM. They are currently using a real-time Model Serving endpoint for both the nightly batch and for the customer-facing chatbot. A cost review shows the nightly batch consumes 70% of the endpoint's token budget. What is the cost optimization recommendation?

- **A)** Increase the Provisioned Throughput max tokens/second to 20,000 — this increases the endpoint's processing speed for the batch job, reducing the time the batch runs and therefore reducing the hourly cost of the endpoint during the batch window.
- **B)** Migrate the nightly batch enrichment from the real-time endpoint to `ai_query()` in a Databricks SQL/Spark pipeline — batch inference via `ai_query()` is more cost-efficient for large-scale non-real-time workloads, freeing the real-time endpoint's capacity and token budget for the customer-facing chatbot.
- **C)** Reduce the batch job to weekly instead of nightly — running 7× less frequently reduces the batch's token consumption proportionally, bringing costs back to the acceptable range without any technical changes.
- **D)** Switch the customer-facing chatbot to a 7B parameter model (from whatever it currently uses) to free up compute capacity for the batch job — right-sizing the chatbot model reduces its per-request cost, leaving more token budget for the nightly batch.

Correct Answer: B **Explanation:** B is correct. The study guide explicitly states: "For non-real-time workloads (e.g., nightly enrichment), use batch `ai_query()` instead of real-time endpoints. Batch inference is generally more cost-efficient for large-scale processing." The current design mixes real-time (customer chatbot) and batch (nightly enrichment) workloads on the same endpoint — this is inefficient: the batch job consumes 70% of the endpoint's budget, competing with real-time traffic. Migrating the nightly batch to `ai_query()` in a Spark/SQL pipeline: (1) Processes 2M records using optimized batch inference (not billed against the real-time endpoint). (2) Frees the real-time endpoint's capacity entirely for chatbot traffic. (3) Allows right-sizing — the batch job can use a smaller, cheaper model if accuracy requirements are lower for classification. A is wrong because increasing Provisioned Throughput max capacity increases the CEILING cost, not reduces it — paying for more reserved capacity when the goal is cost reduction is counterproductive. C is wrong because reducing batch frequency to weekly would significantly delay the data enrichment pipeline — the business requirement is nightly; changing frequency is a business decision, not a technical optimization. D is wrong because right-sizing the chatbot model is a valid optimization, but it doesn't solve the fundamental problem of mixing batch and real-time workloads on the same endpoint. **Source:** Section 6: Evaluation and Monitoring – Objective 4: Use Databricks features to control LLM costs — docs.databricks.com (search: "system.serving.endpoint_usage Databricks")

Question 24

Difficulty: Intermediate

A security-minded developer wants to track every request their LLM agent makes to external tools (web search, database queries) for audit purposes. They also want to track the LLM's reasoning process (what it thought before calling each tool). What MLflow feature captures this at the level of granularity needed?

- **A)** MLflow Experiment run logging — use `mlflow.log_param("tool_calls", json.dumps(tool_call_log))` to log all tool calls as a single parameter string at the end of each agent session, providing an auditable record.
- **B)** MLflow Tracing — each trace captures a SPAN TREE that includes a span for every LLM call (with the input prompt showing the LLM's reasoning), every tool invocation (with inputs and outputs), and retrieved documents. The trace provides a complete, timestamped execution record of everything the agent did.
- **C)** Unity AI Gateway Inference Tables — the Inference Tables log the entire agent session as a single JSON blob containing all tool calls and LLM reasoning in the top-level request/response payload.
- **D)** Databricks Workflows run logs — when the agent runs as a Databricks Workflow, the workflow's run log captures all print statements from the agent code, which can include tool calls if the developer adds print logging.

Correct Answer: B **Explanation:** B is correct. MLflow Tracing is specifically designed for capturing the granular execution detail of multi-step agent workflows. For an agent that calls external tools: (1) **LLM reasoning spans** — each LLM call is captured as a span with its full input (including the chain-of-thought prompt) and output (the LLM's response/plan). This shows the model's reasoning process before each tool call. (2) **Tool invocation spans** — each external tool call (web search, database query) is a child span with: the tool name, input arguments, and returned output, with timing data. (3) **Full span tree** — all spans are organized hierarchically (parent agent call → child LLM call → grandchild tool call), showing the complete execution flow with precise timestamps. This trace is viewable in the MLflow UI and queryable via API, providing the audit trail the developer needs. A is wrong because `mlflow.log_param()` logs static key-value pairs for an ML training run — it is not designed for capturing dynamic, multi-step agent execution flows with timing data. C is wrong because Inference Tables log the TOP-LEVEL request payload and response — they don't automatically capture individual tool calls WITHIN the agent's execution (those require tracing). D is wrong because workflow run logs capture stdout/stderr print output — this is fragile, unstructured, and not designed for auditable tool-call tracking. **Source:** Section 6: Evaluation and Monitoring – Objective 2: Use MLflow and Agent Framework for scoring and tracing — docs.databricks.com (search: "MLflow Tracing agent Databricks")

Question 25

Difficulty: Intermediate

An organization deploys a multi-tenant LLM endpoint serving three business units. Each business unit has a dedicated Service Principal. The Finance team's Service Principal is consuming 80% of the endpoint's total daily token budget, starving the other two teams. What is the correct governance solution using Databricks tooling?

- **A)** Create separate Model Serving endpoints for each business unit — physical endpoint separation is the only way to enforce hard isolation between business unit token budgets.
- **B)** Configure per-Service-Principal rate limits in the Unity AI Gateway for the shared endpoint — set a token/hour quota for the Finance team's Service Principal that is proportional to their fair share (e.g., 33% of total daily capacity), with separate limits for each of the three Service Principals.
- **C)** Increase the Provisioned Throughput max tokens/second — a higher total capacity ensures the Finance team can consume as much as they need without affecting the other teams, because there is more total capacity available.
- **D)** Enable Lakehouse Monitoring on the endpoint — Lakehouse Monitoring automatically detects imbalanced consumption patterns and enforces fair-use policies by throttling the highest consumer without any manual configuration.

Correct Answer: B Explanation: B is correct. Unity AI Gateway Rate Limiting supports per-Service-Principal (per-principal) quota enforcement — the correct tool for this scenario. Configuration: in the Unity AI Gateway UI for the shared endpoint, set token-per-hour quotas for each Service Principal: Finance SP: 167,000 tokens/hour (33% of daily budget), Operations SP: 167,000 tokens/hour, HR SP: 167,000 tokens/hour. When the Finance SP hits its quota, the Gateway returns HTTP 429 for that Service Principal — other SPs continue to receive service. This is the correct architectural pattern for shared LLM endpoints in multi-tenant enterprise environments. A is wrong because separate endpoints is an operationally expensive approach (3× maintenance overhead, 3× monitoring) — per-SP rate limiting on a shared endpoint achieves the same fair-use goal more efficiently. C is wrong because increasing total capacity allows the Finance team to continue consuming 80% at a higher absolute volume — it doesn't solve the fairness problem; it just makes the starvation of other teams happen at a higher cost. D is wrong because Lakehouse Monitoring is for data and model quality monitoring — it detects quality drift and anomalies, not token consumption imbalances. It does not enforce rate limits or throttle consumers. **Source:** Section 6: Evaluation and Monitoring – Objective 7: Use AI Gateway — docs.databricks.com (search: "Unity AI Gateway rate limiting" and "system.serving endpoint usage tables")

Question 26

Difficulty: Intermediate

A developer writes the following custom scorer:

```
from mlflow.genai.scorers import scorer

@scorer
def citation_scorer(inputs, outputs):
    """Score whether the response includes source citations."""
    response = outputs.get("response", "")
    # Check for citation patterns like [1], [Source], (Author, 2024)
    import re
    patterns = [r'\[\d+\]', r'\[Source\]', r'\(\w+, \s*\d{4}\)']
    has_citation = any(re.search(p, response) for p in patterns)
    return 1.0 if has_citation else 0.0
```

Which of the following scenarios correctly identifies a limitation of this scorer?

- **A)** The scorer fails when `outputs` contains a `response` key with an empty string — `outputs.get("response", "")` raises a `KeyError` for empty strings, requiring the developer to add null-checking logic.
- **B)** The scorer returns 0.0 for a response that includes a valid URL citation "For details see: <https://docs.databricks.com/en/generative-ai.html>" — URL-based citations are not matched by any of the three regex patterns, causing valid citations to score as 0.
- **C)** The scorer incorrectly returns 1.0 for a response containing "[2024]" (a year in brackets) — the pattern `r'\[\d+\]'` matches any digits in brackets, not specifically citation numbers.
- **D)** The scorer cannot run in parallel with built-in scorers like `RelevanceToQuery` — custom `@scorer` functions are executed sequentially after all built-in scorers complete in MLflow's evaluation pipeline.

Correct Answer: B **Explanation:** B is correct. This is a practical limitation of the regex-based citation checker: the three patterns are: `r'\[\d+\]'` (matches `[1]`, `[42]`), `r'\[Source\]'` (matches only the exact string `"[Source]"`), and `r'\(\w+, \s*\d{4}\)'` (matches `"(Author, 2024)"`-style citations). A URL citation like `"https://docs.databricks.com/en/generative-ai.html"` does not match any of these patterns — it's not in bracket notation, it's not `"(Name, Year)"` format. So a response with only URL citations scores 0.0, which is incorrect behavior. This is a common pitfall of code-based scorers: they detect only the patterns they were explicitly designed for, missing valid variations. C is also a valid concern worth noting — `[2024]` or `[PDF]` would match `r'\[\d+\]'` and `r'\[Source\]'` respectively — the patterns are not specific enough. However, B is a clearer, more impactful limitation because it produces FALSE NEGATIVES (valid citations scored as 0). A is wrong because `outputs.get("response", "")` correctly returns an empty string (not a `KeyError`) when the key is absent or the value is empty — the code handles this correctly; `re.search` on an empty string simply returns no match and returns 0.0 (correct behavior). D is wrong because custom `@scorer` functions run concurrently with built-in scorers in MLflow's evaluation pipeline — there is no sequential ordering requirement. **Source:** Section 6: Evaluation and Monitoring – Objective 8: Use Databricks custom scorers — [docs.databricks.com \(search: "MLflow custom scorers genai evaluate"\)](https://docs.databricks.com/search/MLflow%20custom%20scorers%20genai%20evaluate)

Question 27

Difficulty: Intermediate

An MLflow trace for a RAG agent's response to "What is the filing deadline for Form 10-K?" shows:

- Span 1 (RETRIEVER): Query → returned chunks about "annual report due dates" (similarity score 0.61)
- Span 2 (LLM): Input included retrieved chunks + user query → Output: "The Form 10-K must be filed within 60 days of fiscal year end for large accelerated filers."
- Span 3 (POST-PROCESSING): Response formatted and returned

A human expert confirms the answer is factually correct. The **RetrievalGroundedness** score is 0.82, **RelevanceToQuery** is 0.91, but the **AnswerCorrectness** score is 0.93. What conclusions can be drawn, and what improvement might be investigated?

- **A)** The groundedness score of 0.82 is dangerously low — a production RAG system must achieve groundedness ≥ 0.95 to be considered safe. The team should immediately retrain the LLM with more legal filings data to improve groundedness.
- **B)** The groundedness score of 0.82 (moderate — the LLM added some facts beyond the retrieved chunks), combined with the low retrieval similarity (0.61), suggests the retriever is returning marginally relevant chunks and the LLM is supplementing with parametric (training) knowledge. Even though the answer is correct, this creates a risk of the LLM hallucinating when its training knowledge is wrong. Investigate: improve chunking or embedding to retrieve higher-similarity (>0.8) chunks that specifically address "Form 10-K filing deadlines."
- **C)** All metrics are positive — groundedness 0.82, relevance 0.91, correctness 0.93 — no investigation is needed. The system is performing within acceptable parameters for all three dimensions simultaneously.
- **D)** The 0.82 groundedness score indicates that 82% of the response is grounded (copied from retrieved chunks) and 18% is hallucinated — since the human expert confirmed correctness, this means 18% of the response contains accurate hallucinations, which is an acceptable production level.

Correct Answer: B **Explanation:** B is correct. This question requires multi-dimensional trace analysis: **Retrieval similarity of 0.61** is relatively low — the chunks about "annual report due dates" are related but not a precise match for "Form 10-K filing deadlines." **Groundedness of 0.82** means the LLM's response is reasonably (but not fully) grounded — some content in the response extends beyond what the retrieved chunks explicitly state. Since the overall answer was correct, the LLM likely supplemented the retrieved context with accurate parametric knowledge from its training. **The risk:** this pattern is dangerous at scale. The current query happened to be answered correctly by the LLM's training knowledge supplementing weak retrieval. For a different query where the training knowledge is outdated or wrong AND retrieval is weak, the LLM will hallucinate confidently. **The investigation:** improve retrieval — better chunking (splitting at section headers for "filing deadlines" sections), better embedding models, or metadata filtering to narrow retrieval to regulatory filing documents. Target similarity scores >0.8 for high-confidence retrieval. A is wrong because there is no "groundedness ≥ 0.95 production threshold" — 0.82 is moderate and warrants investigation but not emergency retraining. C is wrong because while individual metrics look acceptable, the combination of low retrieval similarity + moderate groundedness reveals a systemic risk that warrants improvement. D is wrong because groundedness score is NOT a "percentage of text copied" — it is a holistic quality signal from an LLM judge, not a character-level copying metric. **Source:** Section 6: Evaluation and Monitoring – Objectives 2, 3, 5, 6 — docs.databricks.com (search: "mlflow.genai.evaluate scorers" and "MLflow Tracing agent Databricks")

Question 28

Difficulty: Intermediate

A team needs to compare three LLMs for a multi-turn customer support chatbot. What is the CORRECT process using MLflow experiments?

- **A)** Evaluate all three models in a single MLflow run using `mlflow.evaluate(models=[model_a, model_b, model_c])` — the `models` parameter accepts a list, and MLflow internally runs each model against the dataset and compares results in the experiment UI.
- **B)** Create separate MLflow runs for each model (using `mlflow.start_run(run_name=model_name)` in a loop or separately), run `mlflow.evaluate()` for each, and compare the aggregated metrics across runs in the MLflow Experiment Comparison UI — side-by-side metric comparison enables data-driven model selection.
- **C)** Run `mlflow.compare_models([model_a, model_b, model_c], dataset=golden_dataset)` — this dedicated comparison function runs all three evaluations and automatically generates a comparison report with statistical significance testing.
- **D)** Submit all three models to the MLflow Model Registry with the same alias (e.g., `@champion`), and the Registry automatically runs evaluation experiments and updates the alias to point to the best-performing model.

Correct Answer: B Explanation: B is correct. The standard MLflow experiment-based comparison workflow is:

(1) For each model candidate, create a SEPARATE MLflow run (either explicitly with `mlflow.start_run(run_name="llama-3-70b")` or via loop). (2) Inside each run, call `mlflow.evaluate()` with the same golden dataset and the same set of scorers. (3) MLflow logs all metric results (groundedness, relevance, latency, etc.) to the run. (4) After all runs complete, use the MLflow Experiment UI → Compare Runs feature to view all metrics side-by-side. The developer can sort, filter, and visualize which model performs best across all dimensions. This is the standard MLflow-native approach for model comparison without any custom code beyond `mlflow.start_run()` nesting. A is wrong because `mlflow.evaluate()` does not accept a `models` list parameter — it evaluates ONE model per call. B is the correct multi-run comparison approach. C is wrong because `mlflow.compare_models()` does not exist as an MLflow function — there is no dedicated comparison function with statistical significance testing. D is wrong because MLflow Model Registry alias promotion (to `@champion`) is a deployment decision made by humans after reviewing experiments — the Registry does NOT automatically run evaluations or update aliases based on performance. **Source:** Section 6: Evaluation and Monitoring – Objective 1: Select an LLM based on quantitative metrics — [docs.databricks.com](https://docs.databricks.com/search/MLflow%20evaluate%20generative%20AI%20scorers) (search: "MLflow evaluate generative AI scorers")

Question 29

Difficulty: Intermediate

Agent Monitoring on a deployed legal research assistant shows: for the first 3 months (Jan–Mar), groundedness score averaged 0.88. Starting in April, groundedness drops to 0.71 with high variance. The team hasn't changed the model or the prompt. What is the most likely root cause, and what investigation steps follow?

- **A)** The Provisioned Throughput capacity was exceeded in April, causing the endpoint to queue requests and serve them out of order — queued requests intermix context from different users, reducing groundedness through cross-contamination.
- **B)** User query distribution has shifted — new types of queries may have emerged in April that fall outside the knowledge base coverage (concept drift). Investigate: (1) Query the Inference Table to examine April queries that scored low on groundedness. (2) Check `RelevanceToQuery` scores for the same period — if relevance also dropped, users are asking about topics not covered in the knowledge base. (3) Run a clustering analysis on low-scoring queries to identify the new topic clusters. (4) Update the knowledge base with documents covering the newly emerging topics.
- **C)** MLflow tracing was disabled in April, causing the Agent Monitoring scorer to fall back to a lower-quality heuristic evaluation — the drop in recorded score reflects a measurement artifact, not actual quality degradation.
- **D)** The Unity AI Gateway rate limits were reached in April, causing some requests to return empty responses (HTTP 429) — empty responses score 0.0 on groundedness, dragging the average down.

Correct Answer: B **Explanation:** B is correct. A groundedness drop that starts at a specific time with no model or prompt changes is a classic indicator of CONCEPT DRIFT or QUERY DISTRIBUTION SHIFT. In the context of a legal research assistant, April might coincide with: (1) New legislation passed → users asking about new laws not in the knowledge base. (2) A major court ruling → users asking about case details not yet indexed. (3) A change in the application's marketing/user base → new user profiles asking different types of questions. When users ask about topics not covered in the knowledge base, the retriever returns weakly-related chunks (low similarity scores) and the LLM must supplement from its training knowledge — producing lower groundedness scores. The investigation path: query Inference Tables for low-scoring April traces, examine the queries, cluster them to find new topic patterns, and update the knowledge base. A is wrong because Provisioned Throughput queuing doesn't mix context between users — each request is processed independently; queue delays don't cause groundedness degradation. C is wrong because MLflow tracing being disabled would not cause lower groundedness scores — groundedness scoring runs on the model's inputs/outputs regardless of whether trace data is available. D is wrong because HTTP 429 responses (rate limited) are ERROR responses — they would show as non-200 status codes in the Inference Table, distinguishable from successful but low-quality responses.

Source: Section 6: Evaluation and Monitoring – Objective 5: Use inference tables and Agent Monitoring — docs.databricks.com (search: "Mosaic AI Agent Monitoring" and "Databricks Model Serving endpoint metrics")

Question 30

Difficulty: Intermediate

A developer uses `make_genai_metric()` to create an LLM judge for evaluating medical advice quality. The grading prompt instructs the judge: "Score 1–5. A score of 5 means the advice is evidence-based, mentions limitations, and recommends consulting a doctor." After running evaluation, all 100 test responses receive a score of 5. What is the most likely problem, and how should it be diagnosed?

- **A)** The model is performing perfectly — a mean score of 5 across all 100 responses indicates excellent calibration of the custom scorer with the model's outputs. No investigation is needed.
- **B)** The grading prompt may be insufficiently discriminative — if the criteria are too broad or easy to satisfy, the LLM judge assigns maximum scores to responses that partially satisfy the criteria. Diagnose: (1) Review a sample of the actual responses manually — do they ALL genuinely deserve a 5? (2) Deliberately inject a known-bad response (e.g., no evidence basis, no doctor recommendation) and check if the judge assigns a low score. (3) Refine the grading prompt with specific, harder-to-satisfy criteria and clear rubric examples for each score level (1, 2, 3, 4, 5).
- **C)** The `make_genai_metric()` function has a known ceiling effect bug — when all responses pass a minimum threshold, it rounds all scores to 5. The fix is to use the `@scorer` decorator instead of `make_genai_metric()`.
- **D)** The evaluation dataset is too small — 100 responses are insufficient for statistical significance; with more responses, natural variation will produce a distribution of scores across all 5 levels.

Correct Answer: B **Explanation:** B is correct. A mean score of 5.0 across 100 diverse responses is a red flag — it indicates the LLM judge may have a "sycophancy" problem or the grading criteria are too lenient. Common causes: (1) **Over-general criteria** — "evidence-based, mentions limitations, recommends consulting a doctor" might be easy to satisfy with any medically-worded response. (2) **LLM judge bias** — LLMs used as judges tend toward higher scores unless criteria are very specific. (3) **Grading prompt ambiguity** — the judge may interpret "mentions limitations" too broadly. Diagnosis: (a) Manually review a random sample of responses that scored 5 — do they genuinely deserve 5? (b) INJECT known bad responses (e.g., "Take aspirin for chest pain" — no evidence basis, no doctor recommendation) and verify the judge gives low scores. (c) Refine the rubric with SPECIFIC criteria and EXAMPLE responses for each score level. A is wrong because a perfect uniform score of 5 across 100 varied test cases is statistically suspicious and almost never genuine — any realistic medical response set would have variation in quality. C is wrong because `make_genai_metric()` has no "ceiling effect bug" — the issue is in the grading prompt design, not a library defect. D is wrong because 100 samples is a reasonable evaluation set size — the problem is discriminative validity of the scorer, not statistical sample size. **Source:** Section 6: Evaluation and Monitoring – Objective 8: Use Databricks custom scorers — docs.databricks.com (search: "make_genai_metric Databricks" and "MLflow custom scorers genai evaluate")

Question 31

Difficulty: Advanced

A team builds a medical information RAG system and needs to evaluate it across four dimensions: (1) Does the answer include correct medical information? (2) Is the answer supported by the retrieved medical literature? (3) Is the response written in appropriate clinical language for doctors? (4) Does the response follow safe messaging guidelines (no inappropriate self-treatment advice)? Design the complete MLflow scorer configuration for these four requirements, identifying: which require ground truth, which are built-in vs. custom, and which scorer type for each.

- **A)** All four can be evaluated with built-in scorers: `AnswerCorrectness` for (1), `RetrievalGroundedness` for (2), `Guidelines` for (3), and `Safety` for (4). No custom scorers are needed.
- **B)** (1) `AnswerCorrectness` — REQUIRES ground truth (reference medical answers); built-in. (2) `RetrievalGroundedness` — NO ground truth needed; built-in. (3) Custom LLM Judge (`make_genai_metric()`) with a prompt defining "appropriate clinical language for doctors" — NO ground truth; custom LLM judge. (4) Custom Code-Based Scorer (`@scorer`) that checks for prohibited phrases (e.g., "stop taking your medication") OR a Custom LLM Judge for nuanced safety assessment — NO ground truth; custom scorer. All four run together in `mlflow.genai.evaluate()` .
- **C)** (1) Custom LLM Judge — ground truth is too expensive to create for medical information. (2) `AnswerSimilarity` — compares retrieved chunks to a reference chunk set. (3) `RelevanceToQuery` — clinical language can be inferred from relevance to the clinical query. (4) Built-in `Safety` scorer — safe messaging compliance is a standard safety category.
- **D)** Only (2) and (4) can be automated — (1) medical accuracy requires a doctor (human), and (3) clinical language evaluation requires a clinical linguist (human). All automated scorers should be used only for (2) and (4).

Correct Answer: B **Explanation:** B is correct. Detailed scorer design: (1) **Medical accuracy** →

`AnswerCorrectness` (**ground truth required**): Factual correctness requires a reference answer to compare against. The evaluation dataset must include `expected_output` containing medically accurate reference answers (curated by SMEs). This is the most expensive dimension to evaluate (requires ground truth creation by medical SMEs). (2) **Supported by literature** → `RetrievalGroundedness` (**no GT required**): Checks whether the response claims are traceable to the retrieved medical literature chunks. Compares OUTPUT to RETRIEVED CONTEXT — no reference answer needed. (3) **Clinical language** → **Custom LLM Judge (no GT required)**: "Appropriate clinical language for doctors" is a subjective, nuanced quality that no built-in scorer covers. `make_genai_metric()` with a grading prompt defining: formal medical terminology, avoidance of layman terms, appropriate hedging ("evidence suggests," "consider"), and clinical structure. The LLM judge evaluates this autonomously without a reference answer. (4) **Safe messaging** → **Custom Scorer (no GT required)**: Could be: (a) Code-based: check for prohibited phrases ("stop your medication," "this cures...") — fast and deterministic. (b) LLM judge: for nuanced safe messaging assessment (detecting subtle self-treatment advice). No ground truth needed — evaluated against safe messaging policy. A is wrong because there is no built-in scorer for "clinical language" — Guidelines is close but a custom LLM judge provides much better domain specificity. C is wrong because `AnswerSimilarity` compares to a reference answer, not retrieved chunks; and `RelevanceToQuery` measures relevance, not language register. **Source:** Section 6: Evaluation and Monitoring – Objectives 6 & 8 — docs.databricks.com (search: "MLflow built-in scorers genai" and "MLflow custom scorers genai evaluate" and "make_genai_metric Databricks")

Question 32

Difficulty: Advanced

A production RAG endpoint's Inference Table contains 45 days of data. The team wants to use this data to: (A) Identify the 50 queries where the model performed worst (to create new training examples for fine-tuning), (B) Detect if there are systematic patterns in user queries that the knowledge base doesn't cover, (C) Calculate the average response latency trend over the 45-day period. Write the conceptual SQL approach for each sub-task using the Inference Table and related system tables.

- **A)** (A) `SELECT request_id, response, MIN(groundedness_score) FROM payload_logs GROUP BY request_id, response ORDER BY groundedness_score ASC LIMIT 50;` — groundedness scores are pre-computed and stored in the Inference Table. (B) `SELECT topic_cluster, COUNT(*) FROM payload_logs GROUP BY topic_cluster;` — topic clusters are automatically extracted. (C) `SELECT AVG(latency_ms) FROM payload_logs;` — simple average.
- **B)** (A) Run `mlflow.evaluate()` on the Inference Table data (apply `RetrievalGroundedness` scorer to logged traces) → sort results by groundedness score ascending → extract the 50 lowest-scoring (`request`, `response`) pairs for SME review and fine-tuning annotation. SQL: `SELECT request_id, request, response FROM eval_results_table ORDER BY groundedness_score ASC LIMIT 50`. (B) Extract the `request` column from the payload logs, apply a topic modeling approach (e.g., `ai_query()` or clustering) to group queries by topic, then identify clusters with low groundedness or relevance scores — indicating knowledge gaps. (C) SQL: `SELECT DATE(timestamp) as date, AVG(latency_ms) as avg_latency FROM main.monitoring.payload_logs WHERE timestamp >= CURRENT_DATE - 45 GROUP BY DATE(timestamp) ORDER BY date` — tracks the daily average latency trend.
- **C)** (A), (B), and (C) all require enabling Lakehouse Monitoring — without Lakehouse Monitoring activated on the Inference Table, none of these analyses are possible because raw payload data is not queryable via SQL.
- **D)** (A) Use `mlflow.search_traces(filter_string="metrics.groundedness < 0.5")` — this MLflow API directly filters low-quality traces. (B) The MLflow UI's "Trace Analysis" tab automatically groups traces by topic. (C) The serving endpoint UI provides the latency trend chart without requiring SQL.

Correct Answer: B **Explanation:** B is correct. This is a practical multi-part analysis using the Inference Table: (A)

Finding worst-performing queries: Inference Tables store the raw `request` and `response` payloads but do NOT pre-compute groundedness scores. The team must: (1) Run `mlflow.evaluate()` on a sample of the logged traces (applying the `RetrievalGroundedness` scorer retrospectively to production data). (2) Join the evaluation results (which include per-row groundedness scores) to the payload log records. (3) Sort by groundedness ascending and select the 50 lowest — these are the best candidates for SME review to create fine-tuning examples. (B) **Identifying knowledge gaps:** Topic modeling on the `request` column — use `ai_query()` or a Spark UDF to classify each query's topic, then correlate topic with low quality scores to identify underserved topics. (C) **Latency trend:** Direct SQL aggregation on the `latency_ms` column by day — shows whether latency is increasing over time (might indicate model version changes, traffic growth, or infrastructure issues). A is wrong because groundedness scores and topic clusters are NOT pre-computed columns in the Inference Table — they must be generated by running MLflow evaluators on the logged data. C is wrong because Inference Tables ARE queryable as standard Unity Catalog Delta tables via SQL — no additional Lakehouse Monitoring activation is required for SQL queries against the payload logs. D is partially valid but not complete — `mlflow.search_traces()` is a valid approach for finding low-scoring traces, but the Inference Table SQL approach works for traces already logged there. **Source:** Section 6: Evaluation and Monitoring – Objectives 3, 5, 9 — docs.databricks.com (search: "Inference tables Databricks Model Serving" and "mlflow.genai.evaluate scorers")

Question 33

Difficulty: Advanced

A financial services company has collected 200 production traces from their investment analysis RAG agent. They want an SME (a CFA-certified analyst) to review these traces and provide feedback. The SME is not technical. Design the complete workflow from trace collection through agent improvement.

- **A)** Export all 200 traces as a JSON file and email them to the CFA analyst. The analyst reviews each trace in a JSON viewer and emails back a spreadsheet with their ratings. The developer imports the spreadsheet and manually creates a new evaluation dataset.
- **B)** Step 1: Share the MLflow Review App URL with the CFA analyst — the app requires no coding and presents traces in a readable chat-like interface. Step 2: The analyst reviews each of the 200 traces in "Trace Labeling Mode," rating responses (thumbs up/down or 1–5), adding comments, and optionally providing "expected output" corrections for incorrect responses. Step 3: All feedback is automatically stored as MLflow Assessments. Step 4: The developer uses the Assessments to: (a) Identify systematic failure patterns (types of investment queries with consistently low ratings). (b) Create a new evaluation dataset from SME-labeled (input, expected_output) pairs. (c) Run `mlflow.genai.evaluate()` with `AnswerCorrectness` on this new labeled dataset. (d) Fix identified issues (prompt, chunking, model) and re-evaluate.
- **C)** Have the CFA analyst directly interact with the model serving endpoint API using Postman — the analyst sends test queries and records responses in a spreadsheet. The developer uses the spreadsheet to create a new MLflow experiment comparing current and improved agent versions.
- **D)** The SME feedback workflow requires a Premium MLflow plan — the MLflow Review App and Assessment storage for external SME reviewers (outside the Databricks development team) requires an additional license purchase through the Databricks marketplace.

Correct Answer: B **Explanation:** B is correct. This is the end-to-end SME feedback workflow using Databricks native tooling: **Step 1: Review App** — the MLflow Review App is a web UI accessible via a shareable URL, designed specifically for non-technical SMEs. No Python, no notebooks, no JSON — the SME sees traces in a clean conversational interface. **Step 2: Trace Labeling Mode** — the SME works through the 200 traces, rating each response and providing corrections. For an investment analysis agent, the CFA analyst can correct: "The dividend yield calculation was wrong — it should be $\$1.25 / \$52.30 = 2.39\%$, not 2.4% as stated" (providing the exact expected output). **Step 3: Assessments** — automatically persisted by MLflow with the trace ID, rating, comment, and expected output. **Step 4: Improvement loop** — (a) Systematic pattern analysis: which query types got consistently poor ratings? (b) Labeled dataset: SME-provided `expected_output` corrections become ground truth for `AnswerCorrectness` evaluation. (c) `mlflow.genai.evaluate()` with the labeled dataset quantifies the improvement. (d) Fix root causes → re-evaluate → deploy. A is wrong because JSON/email/spreadsheet is an ad-hoc, fragile approach that loses the trace linkage (which trace triggered which rating) — Assessments maintain this linkage automatically. C is wrong because Postman API testing generates new queries (not reviewing existing production traces) and requires some technical familiarity — it doesn't use the structured Assessment workflow. D is wrong because the MLflow Review App and Assessments are part of the Databricks Mosaic AI platform — no additional "Premium MLflow plan" is required. **Source:** Section 6: Evaluation and Monitoring – Objective 9: Incorporate SME feedback — docs.databricks.com (search: "MLflow Review App human feedback" and "Mosaic AI Agent Evaluation SME feedback")

Question 34

Difficulty: Advanced

A developer monitors a deployed RAG chatbot and observes these Inference Table and Agent Monitoring metrics for a specific 24-hour window:

Metric	Value
Total requests	8,247
4xx error rate	23%
5xx error rate	3%
Average latency_ms	4,200
Groundedness score	0.83
Relevance score	0.89
P99 latency_ms	12,400

Diagnose the three most critical issues and recommend specific Databricks actions for each.

- **A)** Issue 1: 23% 4xx error rate → increase the serving endpoint's concurrency limit. Issue 2: P99 latency of 12,400ms → switch to a faster model. Issue 3: Average latency of 4,200ms → enable caching.
- **B)** Issue 1: 23% 4xx error rate — highly unusual and needs immediate investigation. Likely causes: authentication failures (expired tokens, misconfigured API keys), malformed requests (client-side serialization errors), or rate limiting (Unity AI Gateway returning 429 for users over quota). Action: Query the Inference Table `WHERE status_code BETWEEN 400 AND 499 GROUP BY status_code` to identify which 4xx codes dominate → debug the specific error. Issue 2: Average latency 4,200ms + P99 12,400ms — the extreme P99 indicates tail latency spikes affecting roughly 1% of requests. Action: Query Inference Table for latency outliers, check if they correlate with longer inputs (token-heavy requests), specific times of day (traffic spikes), or specific query types (complex multi-hop retrieval). Consider Provisioned Throughput with autoscaling. Issue 3: 5xx error rate 3% — represents server-side failures on ~250 requests. Action: Check the Model Serving endpoint's infrastructure metrics (CPU/memory) for resource pressure; check Databricks error logs for OOM (out-of-memory) or timeout conditions.
- **C)** The metrics are all within acceptable ranges — a 4,200ms average latency is normal for LLM inference, 23% 4xx rate reflects normal authentication traffic from bots, and 0.83 groundedness is industry-standard. No action required.
- **D)** The dominant issue is groundedness at 0.83 (below the 0.90 minimum production standard). Fix: update the system prompt to "Always cite your sources directly from the retrieved documents." All other metrics are secondary to quality.

Correct Answer: B **Explanation:** B is correct. Systematic diagnostic analysis: **Issue 1 — 23% 4xx error rate (CRITICAL):** A 4xx error rate of 23% is VERY HIGH — nearly 1 in 4 requests is failing with a client-attributable error. For an LLM endpoint, common 4xx causes are: 401/403 (authentication/authorization failure — API tokens expired or permissions changed), 400 (malformed request payload — client bug), 422 (validation error — input schema mismatch), 429 (rate limit hit — Unity AI Gateway throttling). The Inference Table `WHERE status_code BETWEEN 400 AND 499` query reveals the breakdown. 1,897 failed requests in 24 hours is a production-critical incident. **Issue 2 — P99 latency 12,400ms vs. average 4,200ms:** The extreme P99 suggests tail latency is caused by specific request types (long context, complex multi-hop retrieval). This is not a uniform slowdown — it affects ~80 requests. Targeted investigation via Inference Table latency analysis. **Issue 3 — 3% 5xx rate:** ~250 server errors could indicate memory pressure or timeout configuration issues — infrastructure monitoring from the serving endpoint UI. A is wrong because "increase concurrency" doesn't fix authentication failures (4xx); and

"enable caching" is not a direct Databricks serving feature mentioned in the study guide. C is wrong because 23% 4xx is a critical incident — hundreds of real users are experiencing failures per hour. D is wrong because groundedness at 0.83 is a quality concern but not the "dominant issue" — the 23% failure rate is far more urgent. **Source:** Section 6: Evaluation and Monitoring – Objectives 3, 5, 7 — docs.databricks.com (search: "Inference tables Databricks Model Serving" and "Databricks Model Serving endpoint metrics")

Question 35

Difficulty: Proficiency

A healthcare organization wants to build a comprehensive evaluation framework for a clinical decision support RAG system. The system must be evaluated across: accuracy (ground-truth-dependent), safety (automated), clinical language quality (domain-specific), and regulatory compliance (specific required phrases). Design the COMPLETE `mlflow.genai.evaluate()` call with all four scorer types, specifying which require ground truth and the implementation approach for each.

- **A)** All four can use the built-in `Safety` scorer with different `safety_categories` parameters — configure `category = "accuracy"` for clinical accuracy, `category = "safety"` for harm prevention, `category = "clinical"` for language quality, and `category = "compliance"` for regulatory requirements.
- **B)** Complete implementation:

```
from mlflow.genai.scorers import AnswerCorrectness, Safety, scorer, make_genai_metric

# Custom: Regulatory compliance (code-based, deterministic)
@scorer
def compliance_scorer(inputs, outputs):
    required = ["not a substitute for professional medical advice",
                "consult your physician"]
    text = outputs["response"].lower()
    return sum(1 for kw in required if kw in text) / len(required)

# Custom: Clinical language (LLM judge)
clinical_language = make_genai_metric(
    name="clinical_language",
    definition="Response uses formal clinical terminology appropriate for physicians.",
    grading_prompt="Score 1-5: 5=formal clinical terms, precise dosing language, evidence hedging. 1
    model="endpoints:/databricks-meta-llama-3-70b-instruct"
)

results = mlflow.genai.evaluate(
    data=eval_dataset, # Must include expected_output for AnswerCorrectness
    predict_fn=clinical_agent,
    scorers=[
        AnswerCorrectness(), # Requires ground truth
        Safety(),             # No ground truth
        compliance_scorer,    # No ground truth (rule-based)
        clinical_language,    # No ground truth (LLM judge)
    ]
)
```

- **C)** The correct approach is to run four separate `mlflow.evaluate()` calls — one per scorer — because mixing ground-truth-dependent and ground-truth-independent scorers in a single call causes a schema validation error that aborts the entire evaluation.

- **D)** Only `AnswerCorrectness` and `Safety` are available in the Databricks MLflow SDK — for clinical language and regulatory compliance, a separate evaluation service (BioNLP API) must be called and results manually merged with MLflow logs.

Correct Answer: B **Explanation:** B is correct. This is a comprehensive implementation exercise combining all four scorer types in a single `mlflow.genai.evaluate()` call: **(1) `AnswerCorrectness` — REQUIRES ground truth:** Clinical accuracy MUST be checked against expert-curated reference answers. The `eval_dataset` must include `expected_output` column with correct clinical answers (created by medical SMEs). **(2) `Safety()` — NO ground truth:** The built-in Safety scorer evaluates outputs for harmful content (dangerous medical advice, etc.) without needing reference answers. **(3) `compliance_scorer` (Code-Based `@scorer`) — NO ground truth:** Regulatory disclaimers are deterministic — either the required phrase is present or not. Code-based scorer: fast, zero LLM cost, 100% consistent. **(4) `clinical_language` (`make_genai_metric()`) — NO ground truth:** Evaluating whether language is "appropriately clinical for physicians" requires nuanced judgment — an LLM judge with a medical-domain grading prompt is the right tool. MLflow supports mixing ground-truth-dependent and ground-truth-independent scorers in a single call — scorers that need `expected_output` use it when present; scorers that don't need it ignore it. A is wrong because `Safety` does not have a `safety_categories` parameter accepting custom domain names — it evaluates against standard harm categories. C is wrong because MLflow DOES support mixing GT-required and GT-free scorers in one call — they operate independently. D is wrong because custom scorers via `@scorer` and `make_genai_metric()` are available in the Databricks MLflow SDK. **Source:** Section 6: Evaluation and Monitoring – Objectives 6 & 8 — docs.databricks.com (search: "MLflow built-in scorers genai" and "MLflow custom scorers genai evaluate" and "make_genai_metric Databricks")

Question 36

Difficulty: Proficiency

A developer optimizes their RAG chatbot deployment for cost. Current state: Provisioned Throughput endpoint with 5,000 tokens/second reserved, running 24/7. Traffic pattern from `system.serving.endpoint_usage` : 8am–6pm weekdays: 4,200 tokens/sec average. 6pm–8am weekdays + weekends: 180 tokens/sec average. Monthly endpoint cost: \$12,000. Design a complete cost optimization strategy using Databricks features.

- **A)** Cost optimization strategy: (1) Switch from Provisioned Throughput to Pay-per-token entirely — eliminating the hourly reserved capacity cost saves money during off-peak hours. (2) During business hours (8am–6pm weekdays), the pay-per-token endpoint automatically scales. (3) Implement Unity AI Gateway rate limits (4,200 tokens/sec) during business hours to prevent unexpected spikes.
- **B)** Multi-lever cost optimization: (1) Enable Provisioned Throughput AUTOSCALING — set `min_tokens_per_sec = 500` (covers off-peak at 180 tok/s with buffer) and `max_tokens_per_sec = 5,000` (covers peak at 4,200 tok/s). Instead of paying 24/7 for 5,000 tok/s capacity (\$12k/mo), you pay the hourly rate for 500 tok/s during off-peak (nights/weekends $\approx$ 65% of all hours) and scale up during business hours. Estimated savings: 40–50% monthly. (2) Query `system.serving.endpoint_usage` to identify if any off-peak traffic can be moved to batch `ai_query()` (the 180 tok/s off-peak may be automated nightly jobs). (3) Set Unity AI Gateway rate limits per user to prevent runaway queries during peak hours.
- **C)** The correct optimization is to switch all inference to pay-per-token AND enable the Databricks cost optimizer in the Account Console → under "AI Spend Management," enable "Intelligent Cost Routing" which automatically selects the cheapest available model for each request based on token count and complexity.
- **D)** Renegotiate the Provisioned Throughput contract with Databricks account management to get a bulk discount for 12-month commitment — organizational negotiation is the most impactful cost optimization lever, as technical configurations have minimal impact on monthly costs.

Correct Answer: B **Explanation:** B is correct. This is a comprehensive cost optimization analysis: **Traffic pattern analysis:** Peak (8am–6pm weekdays $\approx$ 50 hours/week): 4,200 tok/s. Off-peak (remaining $\approx$ 118 hours/week): 180 tok/s. Currently paying for 5,000 tok/s ALL the time = massive waste during off-peak when only 180 tok/s are used. **Optimization 1 — Autoscaling PT:** Configure min 500 tok/s (covers 180 tok/s with safety margin) and max 5,000 tok/s. During off-peak hours ($\sim$ 65% of time), the endpoint runs at 500 tok/s capacity cost instead of 5,000 tok/s — roughly 10 $\times$ cheaper per hour during off-peak. Estimated savings: if off-peak = 65% of hours, reducing capacity to 10% of peak = $\sim$ 58% savings on off-peak hours = $\sim$ 37% total savings. **Optimization 2 — Identify batch workloads:** Query usage tables to find off-peak automated jobs (180 tok/s at 2am likely includes batch enrichment pipelines). Move these to `ai_query()` batch inference. **Optimization 3 — Rate limits:** Prevent individual power users from consuming peak capacity unnecessarily. A is wrong because pay-per-token completely removes guaranteed capacity — peak hours with 4,200 tok/s demand would face queuing variability; and rate limiting at 4,200 tok/s doesn't prevent spikes above that. C is wrong because "AI Spend Management" and "Intelligent Cost Routing" are not real Databricks features. D is wrong because contract negotiation is not a technical configuration — and technical autoscaling can reduce costs significantly without renegotiation. **Source:** Section 6: Evaluation and Monitoring – Objective 4: Use Databricks features to control LLM costs — docs.databricks.com (search: "system.serving.endpoint_usage Databricks" and "Unity AI Gateway rate limiting")

Question 37

Difficulty: Proficiency

A team has been running their RAG agent in production for 6 months. They collect 500 low-scoring production traces (groundedness < 0.6). They want to use this data to: (1) Create a fine-tuning dataset for improving the LLM, (2) Create an improved evaluation dataset for future testing, and (3) Train a custom LLM judge aligned with their domain. Describe the complete workflow for each, incorporating MLflow Assessments, the Review App, and `mlflow.genai.evaluate()`.

- **A)** All three use cases share the same pipeline: export the 500 traces as JSONL → import to a fine-tuning job on Databricks → the fine-tuned model automatically improves groundedness, evaluation quality, and LLM judge alignment simultaneously.
- **B)** Complete three-stream workflow: **Stream 1 — Fine-tuning dataset:** Deploy the MLflow Review App → have domain SMEs review the 500 low-scoring traces in Trace Labeling Mode → SMEs provide "expected_output" corrections (the right answers the model should have given). Extract `(input, expected_output)` pairs from Assessments → format as fine-tuning dataset (instruction-response pairs) → submit to Databricks Model Fine-Tuning with the curated dataset. **Stream 2 — Improved evaluation dataset:** From the same SME Assessments, extract the high-confidence corrections (where SMEs provided detailed expected_output) as a new GOLDEN DATASET. Use this dataset in future `mlflow.genai.evaluate()` runs with `AnswerCorrectness` — replacing any synthetic or outdated test cases. **Stream 3 — Custom LLM judge training:** Use the SME-labeled Assessments (with human ratings) as preference data to fine-tune a custom judge model OR to calibrate the `make_genai_metric()` grading prompt → verify that the judge's scores correlate with SME ratings on a held-out validation set.
- **C)** The three use cases cannot be executed with the same 500 traces — fine-tuning requires input-output pairs (no SME corrections needed), evaluation datasets require ground truth (SME corrections), and LLM judge training requires preference pairs (SME comparisons). Different trace collections are needed for each.
- **D)** Only Stream 2 (evaluation dataset) is feasible using MLflow tools — fine-tuning (Stream 1) requires the Databricks Data Intelligence Platform's Mosaic AI Fine-Tuning service (separate product), and custom LLM judge training (Stream 3) requires access to the model's gradient backpropagation API.

Correct Answer: B **Explanation:** B is correct. The key insight is that ONE source of SME-labeled Assessments powers ALL THREE use cases: **Source:** 500 low-scoring production traces → SME review via MLflow Review App → Assessments with `(rating, comment, expected_output)`. **Stream 1 — Fine-tuning dataset:** SME-provided `expected_output` corrections = what the model SHOULD have said. These `(input, expected_output)` pairs are exactly the instruction-response format needed for supervised fine-tuning. The model learns from its mistakes on real production data — the most valuable fine-tuning signal. **Stream 2 — Golden evaluation dataset:** High-confidence SME corrections (where SMEs provided detailed, verified expected outputs) become the new ground truth for `AnswerCorrectness` evaluation. This replaces synthetic test cases with real-world failure scenarios — a much more meaningful evaluation set. **Stream 3 — Custom LLM judge calibration:** The human ratings from Assessments reveal how a domain expert evaluates response quality. Use these as: (a) Training signal for a custom judge grader prompt (few-shot examples of "this response got 2/5 because X"). (b) Validation data to verify judge-human agreement on a held-out set. This aligns automated scoring with actual domain expertise. A is wrong because fine-tuning does not automatically improve evaluation quality or train a judge — these are distinct outputs requiring distinct processes. C is wrong because the same Assessments DO serve all three purposes — SME corrections contain `(input, expected_output, rating)` which satisfies all three needs. D is wrong because Mosaic AI Fine-Tuning IS part of the Databricks platform (not a separate product); and LLM judge training via prompt calibration doesn't require gradient access. **Source:** Section 6:

Question 38

Difficulty: Proficiency

An enterprise deploys an agent network: 3 specialized agents (legal, finance, technical) orchestrated by a routing agent. Each agent has its own Model Serving endpoint. The enterprise wants: (1) Complete cost visibility per agent and per department, (2) Quality monitoring for each agent independently, (3) Rate limiting per department (Legal: 100k tokens/day, Finance: 500k tokens/day, Technical: 200k tokens/day). Design the complete monitoring and governance architecture using Databricks tools.

- **A)** (1) Cost: Query `system.serving.endpoint_usage` grouped by `endpoint_name` — each of the 4 endpoints (router + 3 agents) appears as a separate row group, enabling per-agent token cost attribution. Join with department-to-endpoint mapping to get per-department costs. (2) Quality: Enable Agent Monitoring for each of the 3 specialist endpoints — `groundedness`, `relevance`, and `safety` are tracked independently per endpoint, enabling per-domain quality dashboards (legal agent vs. finance agent vs. technical agent). (3) Rate Limits: Configure Unity AI Gateway per-Service-Principal limits for each department's Service Principal on each relevant endpoint. Each department has its own SP — Legal SP: 100k tokens/day on the legal endpoint, Finance SP: 500k tokens/day on the finance endpoint, Technical SP: 200k tokens/day on the technical endpoint.
- **B)** All three requirements (cost, quality, rate limiting) are managed through a single Databricks Feature Store configuration — the Feature Store provides a unified governance layer for cost attribution, quality tracking, and rate limiting for multi-agent systems.
- **C)** (1) Cost: Requires Databricks Premium billing console — standard workspace SQL cannot query cross-endpoint usage. (2) Quality: Run a single `mlflow.evaluate()` on the routing agent's traces — quality of the routing agent implies quality of all specialist agents. (3) Rate Limits: Configure one rate limit on the routing agent endpoint — all downstream specialist agent calls are controlled by the upstream routing limit.
- **D)** Multi-agent systems require a third-party MLOps platform (e.g., Weights & Biases, Comet) for independent per-agent monitoring — the Databricks Mosaic AI Agent Monitoring only supports single-agent deployments and cannot independently track quality metrics for specialist agents in an orchestrated network.

Correct Answer: A **Explanation:** A is correct. This multi-agent governance architecture leverages Databricks native tooling for each requirement: **(1) Cost visibility:** Each endpoint (legal-agent, finance-agent, technical-agent, router-agent) appears as a distinct `endpoint_name` in `system.serving.endpoint_usage`. Query: `SELECT endpoint_name, SUM(total_tokens) FROM system.serving.endpoint_usage GROUP BY endpoint_name ORDER BY total_tokens DESC`. To attribute costs to departments: maintain a mapping table (`endpoint_name` → `department`) and JOIN it with the usage table. Each department's SP calling through their specialized endpoint creates the attribution trail. **(2) Quality monitoring:** Agent Monitoring is configured per-endpoint. Each of the three specialist agents gets its own monitoring dashboard showing groundedness, relevance, and safety over time — legal agent has different quality patterns than finance agent. The router agent gets its own monitoring for routing accuracy. **(3) Rate limiting:** The Unity AI Gateway supports per-Service-Principal rate limits. Each department uses its own SP: Legal SP → 100k tokens/day limit on the legal agent endpoint. Finance SP → 500k tokens/day limit on the finance agent endpoint. Technical SP → 200k tokens/day on the technical agent endpoint. This architecture provides complete isolation, visibility, and control for each domain. B is wrong because Feature Store is for managing ML features (training data), not for monitoring or rate limiting. C is wrong because standard workspace SQL CAN query `system.serving.endpoint_usage` — it's a system table accessible via SQL; and monitoring the router doesn't monitor specialist agents independently. D is wrong because Agent Monitoring supports multi-agent architectures — each agent endpoint is independently monitored. **Source:** Section 6: Evaluation and Monitoring – Objectives 4, 5, 7 — docs.databricks.com (search: "Mosaic AI Agent Monitoring" and "system.serving endpoint usage tables" and "Unity AI Gateway rate limiting")

Question 39

Difficulty: Proficiency

A new product manager asks: "We've been in production for 3 months. Our average groundedness score from Agent Monitoring is 0.81. Our MMLU benchmark score is 76% (which is considered strong). But our customer satisfaction ratings are 3.1/5.0, lower than expected. How do we systematically investigate the gap between strong technical metrics and poor user satisfaction?"

- **A)** The metrics gap is expected — MMLU and groundedness are always higher than user satisfaction because technical metrics measure different dimensions than user experience. No investigation is needed; the 3.1/5.0 satisfaction rate is within industry benchmarks for AI chatbots.
- **B)** The investigation should follow the full evaluation loop: (1) MMLU is an academic benchmark for general knowledge — it does not predict domain-specific performance for your specific application. Discard MMLU as irrelevant for this investigation. (2) Groundedness 0.81 suggests the agent is mostly citing retrieved context, but users are still unsatisfied — the issue is likely quality BEYOND groundedness (relevance, tone, completeness, actionability). (3) Investigate via: (a) Run a comprehensive MLflow evaluation with additional scorers: `RelevanceToQuery` (are answers on-topic?), a custom LLM judge for "actionability" (does the answer help the user take a concrete next step?), and `Guidelines` for tone. (b) Deploy the MLflow Review App for the product's SME team to review the lowest customer-rated sessions — have them annotate what specifically was wrong. (c) Correlate Inference Table query logs with customer satisfaction data (if available) to identify which query types or knowledge base areas correlate with low ratings. (d) Use SME Assessments to create a targeted improvement dataset and re-evaluate.
- **C)** Customer satisfaction ratings below 4.0 indicate the LLM model itself is insufficient — switch immediately to a larger model (GPT-4 or Llama 3 70B) without further investigation. Larger models always correlate with higher user satisfaction.
- **D)** The gap between technical metrics (0.81 groundedness, 76% MMLU) and user satisfaction (3.1/5.0) proves that automated LLM evaluation is fundamentally unreliable — replace all automated scorers with full human evaluation for all future production monitoring.

Correct Answer: B **Explanation:** B is correct. This question tests the ability to connect the full evaluation ecosystem to a real product problem: (1) **MMLU irrelevance** — MMLU is an academic general knowledge benchmark. High MMLU scores don't translate to domain-specific production performance. It should have been used only for initial model shortlisting, not as an ongoing production quality metric. (2) **Groundedness ≠ satisfaction** — A grounded response (citing retrieved context) can still be: unclear or confusing (poor communication), technically correct but not actionable ("The policy allows X" without explaining HOW to apply it), complete but in the wrong tone (too formal/informal for the audience), or missing key context the user wanted. (3) **Investigation path** — The systematic approach uses the full Databricks tooling loop: `RelevanceToQuery` + custom actionability scorer + `Guidelines` scorer in `mlflow.genai.evaluate()` to diagnose WHICH quality dimension is lacking. MLflow Review App for SME feedback to understand qualitatively WHY users are unsatisfied. Inference Table correlation with satisfaction data for quantitative pattern analysis. C is wrong because switching to a larger model without investigation is guessing, not systematic improvement — and satisfaction problems often stem from retrieval quality, prompt design, or knowledge base gaps rather than model capability. D is wrong because the gap between automated metrics and satisfaction doesn't prove automated evaluation is unreliable — it proves that the current scorer selection doesn't capture all dimensions relevant to user satisfaction; the fix is adding better scorers. **Source:** Section 6: Evaluation and Monitoring – Objectives 1, 2, 5, 9 — docs.databricks.com (search: "MLflow evaluate generative AI scorers" and "Mosaic AI Agent Monitoring" and "MLflow Review App human feedback")

Question 40

Difficulty: Proficiency

An organization needs to demonstrate to external auditors that their AI system has been operating safely and ethically for the past year. They must provide: (1) Evidence that all LLM inputs and outputs were monitored for harmful content, (2) Evidence of systematic quality evaluation over time, (3) Evidence that costs were controlled, (4) Evidence that human oversight was maintained. Map each requirement to specific Databricks artifacts, data sources, and processes.

- **A)** (1) Unity AI Gateway Inference Tables (stored in Unity Catalog) — contain timestamped records of every input/output processed; Safety scorer applied retrospectively to production traces proves systematic content monitoring. (2) MLflow Experiment history — contains all evaluation runs with `mlflow.genai.evaluate()` results over time, including metric trends (groundedness, relevance) and comparison across versions. (3) `system.serving.endpoint_usage` query results — show daily/monthly token consumption with rate limit configurations documented in the Unity AI Gateway settings export. (4) MLflow Review App Assessment records — stored assessments prove human evaluators (SMEs) reviewed production traces and provided feedback, with timestamps and reviewer identity.
- **B)** All four requirements are satisfied by enabling the "Compliance Mode" in Databricks Account Settings — Compliance Mode automatically generates quarterly audit reports covering content monitoring, quality evaluation, cost control, and human oversight without any additional configuration.
- **C)** (1) Unity AI Gateway guardrail logs — the gateway automatically creates a signed compliance report for each blocked request. (2) The MLflow Model Registry's version history — every model version update is an implicit evaluation event. (3) The Databricks Account Console billing dashboard — a screenshot of the billing dashboard satisfies cost control evidence requirements. (4) Databricks workspace user activity logs — login timestamps prove human users were active in the workspace, satisfying "human oversight."
- **D)** Evidence cannot be provided from Databricks for any of the four requirements — auditable AI compliance evidence requires a separate certified GRC (Governance, Risk, and Compliance) platform integrated with Databricks via API.

Correct Answer: A **Explanation:** A is correct. Mapping each audit requirement to Databricks evidence artifacts:

(1) Content monitoring evidence — Inference Tables stored in Unity Catalog: provide a permanent, immutable record of every request and response with timestamps. Running the built-in `Safety` scorer via Agent Monitoring or retrospective `mlflow.evaluate()` on Inference Table data demonstrates systematic safety screening. Export these results as audit documentation. **(2) Quality evaluation evidence** — MLflow Experiment history: every `mlflow.genai.evaluate()` run creates a timestamped, versioned record of evaluation metrics. The Experiment Comparison UI shows quality trends across model versions over time — demonstrating systematic evaluation. MLflow run metadata includes: who ran the evaluation, which model version was evaluated, which dataset was used, and what scores were achieved. **(3) Cost control evidence** — `system.serving.endpoint_usage` SQL query results: demonstrate actual token consumption over time. Unity AI Gateway rate limit configurations show proactive controls were implemented. Compare budgeted vs. actual token consumption to demonstrate cost governance. **(4) Human oversight evidence** — MLflow Assessments from the Review App: each Assessment record contains: reviewer identity (username), timestamp of review, trace reviewed, and the rating/correction provided. This creates an auditable trail of human review activity over the audit period. B is wrong because there is no "Compliance Mode" in Databricks Account Settings. C is wrong because login timestamps ≠ meaningful human oversight evidence; and billing screenshots are not sufficient audit artifacts. D is wrong because Databricks native tooling (Inference Tables, MLflow, system tables) provides all four categories of compliance evidence. **Source:** Section 6: Evaluation and Monitoring – Objectives 2, 3, 5, 7, 9 —

docs.databricks.com (search: "Inference tables Databricks Model Serving" and "MLflow Review App human feedback" and "system.serving endpoint usage tables" and "Mosaic AI Agent Monitoring")

Question 41

Difficulty: Proficiency

A developer is tasked with building a fully automated, continuous evaluation pipeline for a production customer support RAG agent. Requirements: (1) Evaluate quality daily using yesterday's production traffic. (2) Alert when groundedness drops below 0.75. (3) Automatically create new evaluation dataset entries when SMEs flag responses as poor. (4) Track cost per successful (non-error) response. Design the complete Databricks implementation.

- **A)** (1) Schedule a Databricks Workflow that runs daily: query yesterday's Inference Table data → run `mlflow.genai.evaluate()` with `RetrievalGroundedness` on the sampled traces → log results to MLflow Experiments. (2) Add a post-evaluation step in the Workflow: `if results.mean("groundedness") < 0.75: send_alert(channel="pagerduty", message="Groundedness alert")` using the Databricks Workflows notification configuration or a webhook. (3) Configure the MLflow Review App for SMEs → when an SME marks a response as "thumbs down" and provides `expected_output`, a Databricks Workflow triggered by new Assessment creation automatically appends the `(input, expected_output)` pair to the golden dataset Delta table. (4) JOIN `system.serving.endpoint_usage` (token counts per `request_id`) with the Inference Table (`status_code` per `request_id`) → filter to `WHERE status_code = 200` → compute `total_tokens / successful_request_count` as cost per successful response.
- **B)** (1) Enable "Auto-Evaluate" mode in the Databricks serving endpoint configuration — this automatically runs daily evaluation without any additional workflow configuration. (2) Databricks natively supports Alert configurations in the endpoint metrics dashboard — set groundedness threshold 0.75 in the monitoring UI. (3) SME feedback automatically updates the golden dataset — the Review App has a "Golden Dataset Auto-Update" toggle. (4) `system.serving.endpoint_usage` natively provides "cost per successful request" as a pre-computed column.
- **C)** (1) Use Lakehouse Monitoring (not `mlflow.evaluate`) for daily quality tracking — Lakehouse Monitoring provides automated drift detection which is equivalent to groundedness monitoring. (2) Lakehouse Monitoring's built-in alerting covers the 0.75 threshold without custom code. (3) SME feedback cannot be automated — human review is always manual. (4) Cost per successful response requires a third-party FinOps tool integration.
- **D)** The pipeline requires a separate MLOps platform — Databricks does not support automated daily evaluation loops, webhook-based alerts, or automated dataset updates from human feedback within a single integrated workflow.

Correct Answer: A **Explanation:** A is correct. This is the most sophisticated integration question — combining the full evaluation and monitoring loop: **(1) Daily evaluation workflow:** A Databricks Workflow (cron-scheduled for daily execution) automates: Query yesterday's Inference Table data (SQL: `WHERE DATE(timestamp) = CURRENT_DATE - 1`) → sample representative traces → run `mlflow.genai.evaluate()` with groundedness scorer on sampled traces → results logged to MLflow Experiments for trend tracking. **(2) Alerting:** In the Workflow, add a Python step that checks `results.metrics["mean/groundedness"] < 0.75` and triggers an alert via webhook (Slack webhook, PagerDuty API, or Databricks Workflow failure notification). **(3) SME feedback → golden dataset automation:** When an SME submits a thumbs-down Assessment with `expected_output` in the Review App, a second Databricks Workflow (triggered by new Assessment records in MLflow storage, or scheduled hourly) queries new Assessments, extracts high-quality `(input, expected_output)` pairs, and appends them to the golden dataset Delta table. **(4) Cost per successful response:** JOIN strategy: `system.serving.endpoint_usage` provides `total_tokens` and `request_id`. Inference Table provides `status_code` and `request_id`. SQL JOIN: `SELECT AVG(u.total_tokens) FROM usage u JOIN payload_logs p ON u.request_id = p.request_id WHERE p.status_code = 200`. B is wrong because "Auto-Evaluate mode,"

"Golden Dataset Auto-Update toggle," and pre-computed "cost per successful request" columns don't exist in Databricks. C is wrong because Lakehouse Monitoring detects data/schema drift but doesn't apply LLM quality scorers (groundedness) — they're complementary tools, not equivalent. D is wrong because all four components are achievable within the Databricks platform. **Source:** Section 6: Evaluation and Monitoring – Objectives 2, 3, 4, 5, 7, 9 — docs.databricks.com (search: "mlflow.genai.evaluate scorers" and "Inference tables Databricks Model Serving" and "system.serving endpoint usage tables" and "MLflow Review App human feedback")

Question 42

Difficulty: Proficiency

After 3 months of production operation, the team reviews their comprehensive monitoring data and wants to prioritize which agent component to improve first. The data shows:

Component	Groundedness	Relevance	P99 Latency	Error Rate
Vector Search Retriever	N/A	0.61	890ms	0.1%
LLM Response Generator	0.79	0.88	2,100ms	0.5%
Re-ranker (post-retrieval)	N/A	0.73	340ms	0.0%
End-to-end pipeline	0.79	0.88	3,400ms	0.6%

Using the principle of systematic root cause analysis, identify the highest-impact improvement and explain the reasoning.

- **A)** Improve the LLM Response Generator first — it has the highest P99 latency contribution (2,100ms) and the most significant error rate (0.5%). Switching to a faster model with lower latency would improve the end-to-end performance most significantly.
- **B)** Improve the Vector Search Retriever first — it has the lowest relevance score (0.61) among measured components, which is the root cause driving downstream quality problems. A retriever relevance of 0.61 means the LLM is receiving marginally relevant chunks, explaining the moderate groundedness (0.79) and end-to-end relevance (0.88). Improving retrieval quality (better chunking, embeddings, or adding a stronger re-ranker) is the highest-leverage improvement because it affects the quality of EVERY subsequent stage. Additionally, at 890ms P99 latency, retrieval is a meaningful latency contributor that optimization (ANN index configuration, caching) could reduce.
- **C)** Improve the Re-ranker first — with a relevance score of 0.73 compared to retriever's 0.61, the re-ranker is underperforming its primary function of improving retrieval quality. A re-ranker that improves relevance by only 12 points (0.61 → 0.73) is contributing minimal value for its 340ms added latency.
- **D)** No improvement is needed — all metrics are within industry-acceptable ranges (groundedness 0.79, relevance 0.88), and the 3,400ms end-to-end latency is typical for RAG pipelines. Optimization should only begin when metrics breach absolute failure thresholds.

Correct Answer: B **Explanation:** B is correct. Root cause analysis: the retriever relevance of 0.61 is the LOWEST score in the entire pipeline and is the ROOT CAUSE of downstream quality limitations. Here's why it's the highest-leverage improvement: **Why retriever is the bottleneck:** In a RAG pipeline, quality flows from retrieval → re-ranking → generation. If retrieval provides low-relevance chunks (0.61), even a perfect LLM cannot produce a perfectly grounded, perfectly relevant response — it's working with poor raw material. The LLM groundedness (0.79) and end-to-end relevance (0.88) both reflect this upstream limitation. **Why re-ranker underperforms:** The re-ranker improves relevance from 0.61 to 0.73 (+0.12), which demonstrates limited effectiveness. This suggests the re-ranker may be poorly tuned for the domain OR that the retriever's chunks are so weakly relevant that even re-ranking can't salvage them. Improving retrieval would give the re-ranker better material to work with. **Why LLM is not the priority:** LLM relevance (0.88) is reasonable given the poor retrieval quality. The LLM is performing its best with what it receives. Switching the LLM for speed (A) doesn't fix the quality problem. **Improvement options for retrieval:** better chunking strategy (smaller, more focused chunks), improved embedding model, metadata filtering, or hybrid search (dense + sparse). A is wrong because while LLM latency is high (2,100ms), addressing latency without fixing quality misses the higher-priority issue — and quality improvements often also improve latency (better retrieval → shorter, more focused context → faster generation). D is wrong because 0.61 retrieval relevance is noticeably low and a clear optimization target. **Source:** Section 6:

Question 43

Difficulty: Proficiency

A large enterprise implements a multi-agent routing system for their internal AI assistant. A master router LLM receives user questions and routes them to one of three specialized sub-agents: HR Agent, IT Support Agent, or Finance Agent. The enterprise needs to independently evaluate the routing accuracy of the master router without penalizing it for incorrect answers generated by the sub-agents. Which evaluation strategy is correct?

- **A)** Use `mlflow.genai.evaluate()` on the entire end-to-end trace, and configure the `AnswerCorrectness` scorer with the parameter `evaluate_routing=True`. This isolates the routing decision and scores it separately from the final generated text.
- **B)** Extract ONLY the inputs (user queries) and the outputs of the router span (the chosen sub-agent name) from the MLflow traces. Create an evaluation dataset with the expected correct sub-agent for each query. Run `mlflow.genai.evaluate()` using `ExactMatch` or a custom Code-Based Scorer to compare the router's choice against the expected choice, ignoring the downstream sub-agent execution completely.
- **C)** Evaluation of routing components requires a separate MLflow Experiment because routing is a classification task, not a generative task. Use `mlflow.sklearn.evaluate()` with accuracy and F1 score metrics, as generative AI scorers cannot evaluate routing logic.
- **D)** Use the MLflow Review App to have SMEs manually review the traces. Automated evaluation is impossible for routing decisions because there is no way to automatically distinguish between a routing error and a generation error in the final output text.

Correct Answer: B **Explanation:** B is correct. Evaluating a router component requires evaluating it as a CLASSIFICATION task, isolated from the downstream generation task. The correct approach: (1) Use MLflow traces to extract the input (user query) and the router's decision (which agent it selected — this is usually the output of the router's LLM span). (2) Build an evaluation dataset containing `(query, expected_agent)`. (3) Evaluate using deterministic metrics like `ExactMatch` (if the expected output is a string like "HR_Agent") or a custom scorer. This isolates the router's performance (routing accuracy) from the sub-agents' performance (answer correctness). A is wrong because `AnswerCorrectness` does not have an `evaluate_routing` parameter — it evaluates the final generated text. C is wrong because `mlflow.genai.evaluate()` CAN evaluate classification-like tasks within gen-AI pipelines using exact match or custom scorers; there is no need to switch to `mlflow.sklearn.evaluate()`. D is wrong because automated evaluation of routing IS possible and highly recommended — you just need the ground truth of the expected route for a set of test queries. **Source:** Section 6: Evaluation and Monitoring – Objectives 2 & 8 — docs.databricks.com (search: "MLflow Tracing agent Databricks" and "MLflow custom scorers genai evaluate")

Question 44

Difficulty: Proficiency

A developer is configuring a custom LLM judge using `make_genai_metric()`. They want to evaluate responses for "Brevity" (how concise the answer is while still being helpful). They test their judge and find it's highly inconsistent — sometimes scoring a 50-word answer a '5', and sometimes a '2'. What is the most effective way to improve the LLM judge's consistency?

- **A)** Increase the `temperature` parameter of the `make_genai_metric()` configuration to 0.7 or higher, allowing the judge model more creativity in how it interprets the grading prompt.
- **B)** Add few-shot examples (demonstrations) to the `grading_prompt` parameter, providing concrete examples of specific responses and the exact score they should receive (e.g., "Example 1: [Text] -> Score: 5. Example 2: [Text] -> Score: 2").
- **C)** Switch the judge model from a Databricks-managed model (like `databricks-meta-llama-3-70b-instruct`) to a smaller custom-tuned model, as smaller models are inherently more deterministic than larger ones.
- **D)** `make_genai_metric` automatically calibrates consistency over time by caching previous responses. The inconsistency will resolve automatically after approximately 100 evaluations as the cache populates.

Correct Answer: B **Explanation:** B is correct. When an LLM judge is inconsistent, the root cause is usually ambiguity in the grading prompt — the judge doesn't have a clear, anchored definition of what a '5' looks like versus a '2'. The most effective technique to improve an LLM judge is **few-shot prompting**: adding concrete examples (demonstrations) of inputs, outputs, and the expected score directly into the `grading_prompt` definition. This anchors the judge's scoring behavior to your specific rubric. A is wrong because increasing `temperature` increases randomness/variance, which makes the judge MORE inconsistent. LLM judges should typically run at low or zero temperature. C is wrong because smaller models are generally WORSE at complex reasoning and instruction following (like adhering to a grading rubric) than larger models like a 70B parameter model. D is wrong because `make_genai_metric` has no automatic "consistency calibration cache" — it evaluates each request statelessly based on the provided prompt and model. **Source:** Section 6: Evaluation and Monitoring – Objective 8: Use Databricks custom scorers — docs.databricks.com (search: "make_genai_metric Databricks")

Question 45

Difficulty: Intermediate

Which of the following scenarios describes a situation where you **MUST** use Pay-per-token billing instead of Provisioned Throughput for a Databricks Model Serving endpoint?

- **A)** The application requires a strict P99 latency SLA of under 1 second during peak business hours.
- **B)** The application uses a custom fine-tuned Llama-3 model registered in Unity Catalog.
- **C)** The endpoint is serving internal development environments where traffic is highly unpredictable and often zero for several days at a time.
- **D)** The endpoint is processing more than 10,000 tokens per second in steady-state production.

Correct Answer: C **Explanation:** C is correct. Pay-per-token is economically mandatory (or highly optimal) for workloads with highly unpredictable, sporadic traffic that often drops to zero (like development, testing, or infrequent batch jobs). Provisioned Throughput charges an HOURLY rate for reserved capacity regardless of whether it's used. If an endpoint has zero traffic for 3 days, Provisioned Throughput still charges you for 72 hours of reserved capacity, resulting in massive waste. Pay-per-token charges \$0 when traffic is zero. A is wrong because strict latency SLAs actually favor Provisioned Throughput, which provides dedicated capacity and avoids queuing delays. B is wrong because both Pay-per-token and Provisioned Throughput support custom fine-tuned models registered in UC. D is wrong because steady-state high-volume production (like >10,000 tokens/sec) is the IDEAL use case for Provisioned Throughput, where the reserved capacity provides predictable latency and often a lower effective per-token cost at high utilization. **Source:** Section 6: Evaluation and Monitoring – Objective 4: Use Databricks features to control LLM costs — docs.databricks.com (search: "Unity AI Gateway rate limiting" and "Foundation Model APIs")

Question 46

Difficulty: Intermediate

An ML engineer is setting up Agent Monitoring for a newly deployed conversational agent. Which two Databricks services must be enabled and properly configured for Agent Monitoring to function and provide quality metrics?

- **A)** Databricks Feature Store and Databricks SQL Serverless.
- **B)** Inference Tables (to capture payload logs) and MLflow Tracing (to capture the execution span tree).
- **C)** Unity Catalog row-level security and Delta Live Tables.
- **D)** Databricks Model Registry and Provisioned Throughput.

Correct Answer: B **Explanation:** B is correct. Agent Monitoring sits on top of two foundational data collection mechanisms: (1) **Inference Tables:** Must be enabled on the serving endpoint to capture the raw incoming requests and outgoing responses (the payloads). (2) **MLflow Tracing:** Must be instrumented in the agent code (via `mlflow.langchain.autolog()`, `@mlflow.trace`, or `agents.deploy()`) so that the internal execution steps (especially retrieved documents for RAG) are captured. Agent Monitoring uses the trace data stored in the Inference Tables to run its automated LLM judges (like computing groundedness by comparing the final response to the retrieved chunks found in the trace). Without Inference Tables, there is no data to monitor. Without Tracing, the monitoring can only evaluate input/output (like Relevance) but cannot evaluate RAG-specific metrics (like Groundedness) because the retrieved chunks aren't visible. A, C, and D are incorrect because they list services that are not dependencies for Agent Monitoring (Feature Store, DLT, Row-level security, etc. are for other data/ML engineering tasks). **Source:** Section 6: Evaluation and Monitoring – Objective 5: Use inference tables and Agent Monitoring — docs.databricks.com (search: "Mosaic AI Agent Monitoring")

Question 47

Difficulty: Intermediate

A company evaluates a generative AI model and finds it scores highly on **AnswerCorrectness** but scores very poorly on **RetrievalGroundedness**. What does this specific combination of metrics indicate about the RAG system's performance?

- **A)** The system is hallucinating false information. The low groundedness indicates the LLM is ignoring the context, and the high correctness is a false positive caused by a misconfigured evaluator.
- **B)** The retriever is working perfectly, but the LLM is failing to synthesize the retrieved information into a coherent answer.
- **C)** The LLM is generating factually correct answers using its own pre-trained parametric knowledge, but it is NOT relying on the retrieved documents. This indicates the retriever is likely failing to find the relevant context, forcing the LLM to guess (and happening to guess correctly).
- **D)** The retrieved documents contain factually incorrect information. The LLM is correctly ignoring the bad retrieved documents and generating the right answer anyway.

Correct Answer: C **Explanation:** C is correct. This is a classic diagnostic scenario. **AnswerCorrectness = High** means the final generated text matches the factual truth (ground truth). **RetrievalGroundedness = Low** means the final generated text is NOT supported by the facts present in the retrieved chunks. If the answer is factually correct but not found in the retrieved context, the LLM must have pulled the correct answer from its own internal training data (parametric knowledge). This is a warning sign: the system is acting as a standalone LLM rather than a RAG system. If it encounters a query where its internal knowledge is weak or outdated, it will hallucinate. The root cause is almost certainly a retrieval failure — the retriever didn't provide the necessary facts, but the LLM answered anyway. A is wrong because high AnswerCorrectness means the information is factually true, not false. B is wrong because if the retriever were perfect, the LLM would likely use it, resulting in high groundedness. D is possible but much less likely than C in a typical enterprise RAG setup; the primary takeaway is that the LLM is not grounded in the provided context, which is risky for enterprise applications relying on private data. **Source:** Section 6: Evaluation and Monitoring – Objectives 2, 5, 6 — docs.databricks.com (search: "mlflow.genai.evaluate scorers")

Question 48

Difficulty: Beginner

A data scientist is reviewing the MLflow Experiment UI after running `mlflow.genai.evaluate()`. They see a metric named `relevance_to_query/v1/mean`. What does the "mean" indicate in this context?

- **A)** The arithmetic average of the relevance scores calculated across all rows in the evaluation dataset.
- **B)** The statistical median of the relevance scores, which MLflow uses to exclude outlier scores.
- **C)** The score of the single worst-performing response in the dataset, used to highlight the "meanest" (most severe) failure.
- **D)** The relevance score computed for the baseline model, used as a comparison point for the candidate model.

Correct Answer: A **Explanation:** A is correct. When `mlflow.genai.evaluate()` runs, it calculates a score for EVERY INDIVIDUAL ROW in the evaluation dataset. It then aggregates these row-level scores to provide summary metrics for the entire run. The `/mean` suffix indicates the arithmetic average of all the individual row scores for that specific metric. (You may also see `/variance` or `/p90` depending on the configuration). This mean value is what you typically use to compare overall model performance between different MLflow runs. B is wrong because mean is the arithmetic average, not the median. C and D are nonsensical interpretations of the term "mean" in statistics and MLflow. **Source:** Section 6: Evaluation and Monitoring – Objective 2: Use MLflow and Agent Framework for scoring and tracing — docs.databricks.com (search: "mlflow.genai.evaluate scorers")

Question 49

Difficulty: Proficiency

Your enterprise requires that all LLM interactions be completely traceable, including the exact prompt sent to the model, the exact retrieved chunks, and the latency of each step. You are using a custom Python framework (NOT LangChain or LlamaIndex) to build your agent. How do you implement this tracing requirement in Databricks?

- **A)** You must rewrite the agent using LangChain and use `mlflow.langchain.autolog()`, because custom Python frameworks cannot be traced by MLflow.
- **B)** Use the `@mlflow.trace` decorator on your main agent function, and also apply it to the specific helper functions that perform retrieval and LLM API calls. Use the `span_type` parameter to categorize the spans (e.g., `span_type="RETRIEVER"`, `span_type="LLM"`).
- **C)** Enable Inference Tables on the serving endpoint. Inference Tables automatically parse custom Python code and generate execution span trees without any code changes.
- **D)** Use `mlflow.log_text()` at every step of your Python code to write the inputs, outputs, and latencies to text files attached to the MLflow run.

Correct Answer: B **Explanation:** B is correct. MLflow Tracing is framework-agnostic. While `autolog()` provides zero-code instrumentation for supported frameworks like LangChain, custom Python code can be fully instrumented using the `@mlflow.trace` decorator. By decorating the main function (creates the root span) and the nested helper functions (creates child spans), you build a complete span tree. The `span_type` parameter (e.g., "RETRIEVER", "LLM", "TOOL", "CHAIN") tells the MLflow UI how to render and categorize the span, enabling the same rich visualization and evaluation capabilities as framework-based agents. A is wrong because rewriting is unnecessary; MLflow supports custom code tracing. C is wrong because Inference Tables capture the top-level request/response payload; they do NOT automatically introspect custom Python execution to build internal span trees. Tracing must be explicitly instrumented in the code. D is wrong because `mlflow.log_text()` just dumps unstructured text files to an ML training run; it does not create the structured, queryable, hierarchical trace objects required for Agent Evaluation. **Source:** Section 6: Evaluation and Monitoring – Objective 2: Use MLflow and Agent Framework — docs.databricks.com (search: "MLflow Tracing agent Databricks")

Question 50

Difficulty: Beginner

Which of the following is NOT a metric typically used to evaluate the RETRIEVAL step of a RAG pipeline?

- **A)** Recall@k (Are the relevant documents present in the top k results?)
- **B)** Precision@k (What proportion of the top k results are actually relevant?)
- **C)** NDCG (Normalized Discounted Cumulative Gain - Is the ordering of the retrieved results optimal?)
- **D)** TTFT (Time to First Token)

Correct Answer: D **Explanation:** D is correct because TTFT (Time to First Token) is a metric used to evaluate the GENERATION step (the LLM's streaming latency), not the RETRIEVAL step. Retrieval metrics focus on whether the search component found the right information and ranked it correctly. Recall@k measures if the needed facts were retrieved at all within the 'k' chunks sent to the LLM. Precision@k measures how many of the retrieved chunks were actually useful (vs. noise). NDCG measures ranking quality (are the most relevant chunks at the very top). TTFT has nothing to do with search quality; it only measures how fast the LLM starts generating text after receiving the prompt. **Source:** Section 6: Evaluation and Monitoring – Terminology Breakdown & Objective 1 — docs.databricks.com (search: "Foundation Model APIs model comparison")

Question 51

Difficulty: Intermediate

An ML engineer is analyzing system usage tables to allocate LLM costs to different departments. They run the following query:

```
SELECT
    principal_name,
    SUM(total_tokens) as tokens
FROM system.serving.endpoint_usage
WHERE timestamp > '2024-01-01'
GROUP BY principal_name
```

They notice a large percentage of tokens are attributed to a `principal_name` representing a generic service account used by multiple backend jobs, making it impossible to distinguish which department is responsible for those costs. What architectural change is required to enable accurate department-level cost attribution?

- **A)** The engineer needs to parse the Inference Table payload logs to read the user's name from the chat interface, and join that with the usage table.
- **B)** The enterprise must provision separate, dedicated Service Principals for each department's backend jobs, and configure those jobs to authenticate with their respective Service Principal when calling the Unity AI Gateway.
- **C)** The engineer should enable the `department_id` parameter in the Unity AI Gateway configuration, which forces all users to select their department from a dropdown before the request is processed.
- **D)** The enterprise must create a separate Databricks workspace for each department, as system tables cannot differentiate costs within a single workspace.

Correct Answer: B **Explanation:** B is correct. In Databricks, the `principal_name` in the `system.serving.endpoint_usage` table corresponds to the identity (User or Service Principal) that authenticated the API request to the serving endpoint. If multiple applications or departments share a single generic Service Principal (e.g., `app-backend-sp`), all their usage aggregates under that single identity, destroying attribution granularity. The standard architectural best practice for FinOps and governance is to provision a dedicated Service Principal for each application or department (e.g., `finance-app-sp`, `hr-app-sp`). When each application authenticates with its own SP, the usage table correctly attributes token consumption to the specific principal, enabling accurate chargebacks. A is wrong because payload parsing is fragile, often PII-sensitive, and doesn't work for automated backend jobs that don't pass a "user name." C is wrong because there is no `department_id` prompt parameter in the AI Gateway. D is wrong because separate workspaces are massive overkill and create administrative nightmares; dedicated Service Principals within a single workspace provide the exact isolation needed. **Source:** Section 6: Evaluation and Monitoring – Objective 7: Use AI Gateway — docs.databricks.com (search: "system.serving endpoint usage tables")

Question 52

Difficulty: Proficiency

A development team is preparing to deploy an agent to production. They have established a golden dataset and run `mlflow.genai.evaluate()`. The results show acceptable quality scores. However, the security team requires proof that the model is resilient against jailbreak attempts before approving the deployment. How should the team provide this evidence using Databricks?

- **A)** Use the MLflow Review App to manually type 50 known jailbreak prompts into the agent's chat interface and record screen captures proving the agent refuses to answer.
- **B)** Configure the Unity AI Gateway to block all requests containing the word "ignore", which technically satisfies the security team's requirement without needing evaluation.
- **C)** Create a specialized evaluation dataset containing malicious inputs (jailbreak attempts, prompt injections). Run `mlflow.genai.evaluate()` on this dataset using the built-in `Safety` scorer or a custom LLM judge designed to detect successful jailbreaks (e.g., scoring whether the model erroneously complied with the malicious instruction). Provide the resulting MLflow evaluation metrics as evidence.
- **D)** This cannot be done offline. The team must deploy the agent, wait for real users to attempt jailbreaks, and then use Agent Monitoring to show the security team how the system reacted in production.

Correct Answer: C **Explanation:** C is correct. Security testing (red teaming) for LLMs should be treated as a specialized evaluation workflow. The correct approach is to build a golden dataset of *adversarial* inputs (jailbreaks, prompt injections). You then run `mlflow.genai.evaluate()` on this adversarial dataset. You evaluate the outputs using the built-in `Safety` scorer (which checks for harm) or, more accurately for jailbreaks, a custom LLM judge (`make_genai_metric`) whose grading prompt asks: "Did the model comply with the malicious instruction or did it safely refuse?" The resulting MLflow Experiment run provides quantitative, auditable evidence of the model's resilience. A is wrong because manual testing is not scalable, repeatable, or easily auditable compared to a reproducible MLflow evaluation run. B is wrong because keyword blocking (like "ignore") is a primitive, easily bypassed filter that breaks legitimate queries (e.g., "ignore case in this regex") and does not prove model resilience. D is wrong because security testing **MUST** be done offline (pre-deployment) to satisfy security gates; waiting for production attacks is irresponsible. **Source:** Section 6: Evaluation and Monitoring – Objective 2: Use MLflow and Agent Framework — docs.databricks.com (search: "mlflow.genai.evaluate scorers")

Question 53

Difficulty: Intermediate

Which of the following describes the correct relationship between a trace, a span, and an assessment in the Databricks Mosaic AI ecosystem?

- **A)** A trace represents a single evaluation run, a span represents the dataset used, and an assessment represents the final MLflow metrics.
- **B)** A span represents a single operation (like an LLM call), a trace is the hierarchical collection of all spans for a single request, and an assessment is a human evaluator's qualitative feedback attached to that specific trace.
- **C)** An assessment is the automated score generated by an LLM judge, a trace is the human feedback, and a span is the amount of time the request took.
- **D)** A trace logs the token usage, a span enforces the rate limit, and an assessment determines the billing cost.

Correct Answer: B **Explanation:** B is correct. This tests fundamental telemetry terminology. A **span** is the fundamental unit of execution logging — it represents a single, timed operation (e.g., "Retrieve from Vector Database", "Call Llama-3"). A **trace** is the complete, hierarchical tree of all spans associated with a single top-level user request (from input to final output). An **assessment** is a piece of human feedback (rating, comment, correction) generated in the MLflow Review App (or via API) that is explicitly linked to a specific trace ID, indicating human judgment on that specific execution. A, C, and D mix up these definitions with unrelated concepts like billing, evaluation runs, or automated scoring. **Source:** Section 6: Evaluation and Monitoring – Objectives 2 & 9 — docs.databricks.com (search: "MLflow Tracing agent Databricks" and "MLflow Review App human feedback")

Question 54

Difficulty: Intermediate

You are designing a code-based custom scorer (`@scorer`) to evaluate whether a generated SQL query contains any destructive operations (e.g., DROP, DELETE, TRUNCATE). You want the scorer to return `0.0` if destructive operations are found, and `1.0` if it is safe. The inputs are `inputs` (dict containing the user question) and `outputs` (dict containing the model's response under the key "query"). Which of the following is the correct implementation?

- **A)** `python @scorer def safe_sql_scorer(inputs, outputs): sql = outputs.get("query", "").upper() if "DROP" in sql or "DELETE" in sql or "TRUNCATE" in sql: return 0.0 return 1.0`

```
* **B)** python
make_genai_metric(
    name="safe_sql",
    definition="Check for DROP, DELETE, or TRUNCATE",
    model="exact_match_code"
)
```

- **C)** `python @scorer def safe_sql_scorer(inputs, outputs): return mlflow.sql.evaluate(outputs["query"], check_destructive=True)`

```
* **D)** python
def safe_sql_scorer(inputs, expected_outputs):
    if expected_outputs == "DROP": return 0.0
    return 1.0
```

Correct Answer: A **Explanation:** A is correct. A code-based custom scorer is implemented using the `@scorer` decorator from `mlflow.genai.scorers`. The function must accept `inputs` and `outputs` as dictionaries. Option A correctly extracts the generated string from `outputs`, converts it to uppercase for case-insensitive matching, checks for the forbidden SQL keywords, and returns the appropriate float score (0.0 for unsafe, 1.0 for safe). This is a deterministic, fast execution requiring no LLM. B is wrong because `make_genai_metric` is used for creating LLM-based judges (using a grading prompt), not code-based exact match rules. C is wrong because `mlflow.sql.evaluate` does not exist. D is wrong because it lacks the `@scorer` decorator, has the wrong signature (missing `outputs`), and compares against `expected_outputs` instead of parsing the actual generated text. **Source:** Section 6: Evaluation and Monitoring – Objective 8: Use Databricks custom scorers — docs.databricks.com (search: "MLflow custom scorers genai evaluate")

Question 55

Difficulty: Proficiency

A Databricks customer notices that their Pay-per-token LLM endpoint frequently experiences high latency (TTFT > 3 seconds) during the hours of 9:00 AM to 11:00 AM, but operates normally (TTFT < 0.5 seconds) the rest of the day. They want to ensure predictable, sub-second TTFT during these morning hours without overspending. What is the optimal solution?

- **A)** Continue using Pay-per-token but increase the Unity AI Gateway rate limits for the affected users during the morning hours.
- **B)** Switch the endpoint to Provisioned Throughput with autoscaling. Configure the `min_tokens_per_sec` to handle the normal baseline traffic, and set `max_tokens_per_sec` high enough to absorb the 9:00 AM - 11:00 AM peak.
- **C)** Create a Databricks Job that restarts the Model Serving endpoint every morning at 8:50 AM to clear the cache and reset the latency metrics.
- **D)** Switch to a smaller model exclusively during the 9:00 AM to 11:00 AM window using a time-based routing script.

Correct Answer: B **Explanation:** B is correct. The customer is experiencing the primary drawback of Pay-per-token billing: noisy neighbor or capacity queuing during peak usage times, resulting in unpredictable latency spikes. To guarantee predictable latency (SLA), you must reserve dedicated capacity using **Provisioned Throughput**. However, reserving peak capacity 24/7 is expensive. The optimal Databricks solution is Provisioned Throughput with **autoscaling**. By setting a minimum capacity for baseline hours and a high maximum capacity for the peak hours, Databricks automatically scales the underlying compute to handle the morning spike without queuing delays, and scales back down afterward to save costs. A is wrong because rate limits control who can access the endpoint, but they don't add underlying compute capacity to a shared Pay-per-token pool. C is wrong because restarting an endpoint causes downtime and doesn't solve capacity constraints. D is wrong because swapping to a smaller model changes the quality of the application drastically depending on the time of day, which is generally an unacceptable user experience. **Source:** Section 6: Evaluation and Monitoring – Objective 4: Use Databricks features to control LLM costs — [docs.databricks.com \(search: \"Foundation Model APIs model comparison\"\)](https://docs.databricks.com/search/\)

Question 56

Difficulty: Beginner

When running `mlflow.genai.evaluate()`, what is the primary purpose of an evaluation dataset (often called a golden dataset)?

- **A)** To provide the training data necessary to fine-tune the LLM prior to evaluation.
- **B)** To provide a standardized set of representative inputs (and optionally expected outputs) that can be repeatedly used to measure and compare model performance objectively over time.
- **C)** To store the final logs and metrics output generated by the evaluation process for auditing purposes.
- **D)** To define the rate limit quotas and access controls for the model serving endpoint.

Correct Answer: B **Explanation:** B is correct. An evaluation dataset (or golden dataset) is a curated collection of records used to test the model. In MLflow, this is typically a Pandas DataFrame, Spark DataFrame, or list of dictionaries containing `inputs` (the questions/prompts) and optionally `expected_output` (the reference correct answers) and `context` (retrieved chunks, if testing offline without a live retriever). The primary purpose is to provide a consistent, repeatable benchmark. By running different models or different prompt versions against the EXACT SAME golden dataset, developers can objectively measure if a change improved or degraded performance. A is wrong because evaluation datasets are for TESTING, not training/fine-tuning. C is wrong because the dataset is the INPUT to evaluation, not the output logs (output metrics are stored in MLflow runs/experiments). D is wrong because rate limits are configured in the AI Gateway, not in an evaluation dataset.

Source: Section 6: Evaluation and Monitoring – Terminology & Objective 2 — docs.databricks.com (search: "mlflow.genai.evaluate scorers")

Question 57

Difficulty: Intermediate

An enterprise wants to evaluate a chat model's ability to maintain a coherent persona and follow instructions over a long, multi-turn conversation. Which academic benchmark is specifically designed to measure this capability?

- **A)** HumanEval
- **B)** MMLU (Massive Multitask Language Understanding)
- **C)** MT-Bench
- **D)** GSM8K

Correct Answer: C **Explanation:** C is correct. **MT-Bench** (Multi-Turn Benchmark) is specifically designed to evaluate the ability of LLMs to engage in multi-turn conversations and follow complex instructions across a dialogue. It asks an initial question, and then follows up with a second question that depends on the context of the first, requiring the model to maintain state and persona. A is wrong because HumanEval evaluates code generation (Python functions). B is wrong because MMLU evaluates broad general knowledge (answering multiple-choice questions across 57 subjects) in single-turn prompts. D is wrong because GSM8K evaluates multi-step grade-school mathematical reasoning. **Source:** Section 6: Evaluation and Monitoring – Objective 1: Select an LLM based on quantitative metrics — docs.databricks.com (search: "Foundation Model APIs model comparison")

Question 58

Difficulty: Proficiency

Your team has built a Custom LLM Judge using `make_genai_metric()` to score the "Tone" of customer service responses. You run this judge over your golden dataset and get the results. What is the BEST practice for validating that your Custom LLM Judge is actually scoring responses correctly?

- **A)** Run the built-in `AnswerCorrectness` scorer alongside your custom judge and ensure the two scores are identical for every row.
- **B)** Take a statistically significant sample of the responses, have a human domain expert (SME) manually score them for Tone using the same 1-5 scale, and calculate the correlation (e.g., Pearson or Spearman) between the LLM Judge's scores and the human SME's scores.
- **C)** Evaluate the Custom LLM Judge against the MT-Bench academic benchmark to prove its general reasoning capabilities.
- **D)** Check that the variance of the Custom LLM Judge's scores is exactly 0.0, indicating perfect consistency.

Correct Answer: B **Explanation:** B is correct. A Custom LLM Judge is itself a model, and its grading logic must be validated against human judgment (the ultimate ground truth for subjective metrics like Tone). The industry best practice is to have human Subject Matter Experts (SMEs) label a subset of the data (using a tool like the MLflow Review App) and then measure the correlation or agreement rate between the human scores and the LLM Judge scores. If correlation is high (e.g., >0.8), the LLM Judge is well-aligned with human experts and can be trusted to grade responses at scale automatically. If correlation is low, the judge's grading prompt needs refinement. A is wrong because `AnswerCorrectness` measures factual accuracy, not Tone; a response can be factually correct but have terrible tone, so the scores should NOT be identical. C is wrong because MT-Bench tests chat capability, not grading capability; you must validate the judge on your specific domain rubric. D is wrong because a variance of 0.0 means the judge gave the exact same score to every response, which usually indicates a broken grading prompt (sycophancy or inability to discriminate quality), not perfect accuracy. **Source:** Section 6: Evaluation and Monitoring – Objectives 8 & 9 — docs.databricks.com (search: "make_genai_metric Databricks" and "Mosaic AI Agent Evaluation SME feedback")

Question 59

Difficulty: Beginner

When analyzing MLflow Tracing data for a RAG application, which span typically contains the `similarity_score` metric, indicating how closely a retrieved document matches the user's query?

- **A)** The top-level `CHAIN` span.
- **B)** The `LLM` span.
- **C)** The `RETRIEVER` span.
- **D)** The `POST_PROCESSING` span.

Correct Answer: C **Explanation:** C is correct. In a standard MLflow trace for a RAG agent, the `RETRIEVER` span represents the execution of the vector search or document retrieval function. The output of this span typically contains the list of retrieved document chunks, along with their associated metadata, including the `similarity_score` (or distance metric) returned by the Vector Search index. The top-level `CHAIN` (A) represents the entire request. The `LLM` span (B) contains the generation prompt and the model's text response. The `POST_PROCESSING` span (D) handles formatting. If you need to debug poor retrieval quality (e.g., low similarity scores), you inspect the `RETRIEVER` span in the trace. **Source:** Section 6: Evaluation and Monitoring – Objective 2: Use MLflow and Agent Framework — docs.databricks.com (search: "MLflow Tracing agent Databricks")

Question 60

Difficulty: Intermediate

An ML engineer wants to implement a Databricks system that automatically blocks generation if the user's prompt contains a known prompt injection attack. Which architectural layer is the MOST appropriate place to enforce this rule before the request ever reaches the expensive LLM?

- **A)** In an MLflow Custom Scorer (`@scorer`) that runs during the nightly evaluation batch.
- **B)** Inside the Inference Table schema definition using Delta constraints.
- **C)** In the Unity AI Gateway by configuring input guardrails or a pre-generation safety filter.
- **D)** By manually analyzing the Agent Monitoring dashboard and blocking users retroactively.

Correct Answer: C **Explanation:** C is correct. Guardrails that PREVENT malicious inputs (like prompt injections or jailbreaks) from reaching the LLM must be enforced in the routing/gateway layer. The Unity AI Gateway is designed to sit between the client application and the LLM endpoint. Configuring input guardrails or safety filters at the AI Gateway ensures the request is intercepted, analyzed, and blocked (returning an error to the user) BEFORE an expensive LLM inference call is made. A is wrong because MLflow custom scorers evaluate quality AFTER the fact (offline or asynchronously); they do not block real-time traffic. B is wrong because Inference Tables log data AFTER it happens; Delta constraints cannot block a live API request to an LLM. D is wrong because retroactive analysis does not block the attack from executing in the first place. **Source:** Section 6: Evaluation and Monitoring – Objective 7: Use AI Gateway — docs.databricks.com (search: "Unity AI Gateway rate limiting" and "Databricks AI Gateway guardrails")